VRIJE UNIVERSITEIT AMSTERDAM

UNIVERSITEIT VAN AMSTERDAM

Master Thesis

Selective Evidence-Aware LLM Fault Localization Across Benchmark and Real-World Repositories

by

Ziniu Jin

First supervisor: Ana Oprescu

August 20, 2026

Submitted in partial fulfillment of the requirements for
the joint UvA-VU degree of Master of Science in Computer Science

Selective Evidence-Aware LLM Fault Localization Across Benchmark and Real-World Repositories

Ziniu Jin
Vrije Universiteit Amsterdam
Amsterdam, The Netherlands

ABSTRACT

[Ana review: completed] The abstract has been broadened to foreground the problem, method family, evaluation scope, and main findings without reproducing detailed result tables or implementation settings.

File-level fault localization helps developers decide which source files to inspect after a failure. Retrieval-based methods are inexpensive and reproducible, but they can miss evidence carried by failing tests, stack traces, and local source context. Large language models can use richer semantic evidence, but applying them to every bug increases calls, tokens, runtime, and reproducibility risk. This thesis studies selective evidence-aware reranking as a cost-controlled middle ground for repository-level debugging support.

The method first builds a bounded candidate pool, constructs structured bug and source-code evidence, and uses a non-oracle selector to decide which cases receive one-shot reranking. Non-selected cases keep the retrieval ranking through fallback. The evaluation combines frozen Defects4J validation, including a follow-up on a previously unused project, with a company bug-log case study and a retrospective git-history-derived case study. The results show that selective reranking can improve file-level ranking while using fewer model calls than reranking every case, but also that reranking all cases is more accurate under the same evidence package. The method should therefore be interpreted as an accuracy–cost trade-off rather than as the highest-accuracy configuration. The main bottlenecks are candidate recall, selector recall, and evidence quality; under the evaluated controlled settings, additional agentic or verifier calls do not consistently improve file-level ranking.

Contents

1 INTRODUCTION

[Ana review: completed] The thesis has been restructured so that the research questions appear in the Introduction before the Design, Evaluation, and Results sections.

Debugging often stalls not because tests cannot expose a failure, but because the next file to inspect is unclear. Fault localization addresses this problem by ranking program elements according to their likelihood of containing the fault. Prior work covers spectrum-based methods, statistical models, and learning-based approaches [1–5]. This thesis studies a practical file-level setting: given a bug record and a buggy repository snapshot, the system returns a ranked list of suspicious source files. File-level localization is coarser than statement-level diagnosis, but it matches an early debugging decision that developers and automated repair systems both need to make.

Information-retrieval-based bug localization is a natural baseline for this setting because it is reproducible, inexpensive, and compatible with bug reports and source-code indexes. BM25-style retrieval provides a strong lexical ranking model for text collections [6], and prior IR-based bug localization systems use bug reports, source-code text, structured fields, code-change histories, and project history to rank likely faulty files [7–9]. These methods scale to whole repositories without asking a model to read every file. Their weakness appears when the bug report and relevant code use different vocabulary, or when the decisive signal appears in failing tests, stack traces, or local snippets rather than in the report text. In those cases, lexical ranking can leave the correct file too low for practical inspection.

Large language models bring a different capability: they can relate code, error messages, and natural-language bug context. Recent studies show that LLMs can help with fault localization, but also that performance depends strongly on context size, prompt design, localization granularity, and code-model representation choices [10–15]. Project-level and agentic systems address repository context by letting an LLM inspect code over multiple steps [16–19]. That flexibility has a cost: whole-repository prompting and multi-step inspection add latency, token usage, and extra failure modes. For file-level ranking, the central question is therefore not whether an LLM can inspect code in principle, but when it is worth invoking the LLM and what evidence it should receive.

This thesis studies selective evidence-aware LLM reranking for file-level fault localization. The method first retrieves a candidate pool with BM25 and focused hybrid retrieval. It then constructs deterministic evidence from the bug context, failing tests, stack traces, source-code snippets, and candidate metadata. A non-oracle selector decides whether a bug appears difficult enough for LLM reranking. Selected cases are sent to a one-shot DeepSeek reranker; non-selected cases keep the retrieval ranking through fallback. The design treats retrieval as the recall and cost-control stage, and the LLM as a targeted reranker rather than a replacement for repository search.

1.1 Task Definition and Scope

Given a bug record and a buggy repository snapshot, the system returns a ranked list of suspicious source files. The bug record may contain an issue description, failure message, triggering test, stack trace, or other reproduction context, while the repository snapshot contains the source files available before the fix is applied. The file-level granularity represents an early debugging decision: which files should a developer or downstream repair system inspect first?

The scope is intentionally narrower than debugging or automated repair. The system does not generate patches, prove root causes, or identify faulty statements. Defects4J provides controlled benchmark evidence, while AboutWork and Easy Finance are treated as repository case studies rather than broad industrial validation. Easy Finance uses retrospective commit-derived query text and therefore does not provide the same prospective input boundary as Defects4J or AboutWork. Across all datasets, repair diffs, changed-file labels, and post-fix source code remain excluded from prediction; the detailed ground-truth and no-leakage rules are defined in the Evaluation Setup.

1.2 Research Questions

The thesis is organized around three research questions. They connect the related-work gap to the evaluation: whether selective evidence-aware reranking improves file-level localization, why its accuracy–cost trade-off succeeds or fails, and how controlled agentic or verifier extensions compare with one-shot reranking.

RQ1 To what extent does selective evidence-aware LLM reranking improve file-level fault localization over retrieval-only baselines on benchmark and real-world repository data?

RQ2 How do candidate recall, evidence quality, and non-oracle selection affect the accuracy–cost trade-off of selective LLM reranking?

RQ3 How do controlled agentic inspection and verifier reranking compare with one-shot LLM reranking in accuracy, cost, and failure modes?

Benchmark results are treated as controlled validation evidence, while AboutWork and Easy Finance are interpreted as real-world case studies rather than broad industrial generalization. Agentic inspection and verifier reranking are evaluated as controlled extensions and ablations, not as the primary thesis method.

The evaluation is intentionally conservative. The main benchmark evidence comes from the frozen Closure 61–100 held-out slice of Defects4J [20]. This slice contains 38 usable records because Closure-63 and Closure-93 are absent from the active-bug metadata; it is the strongest held-out subset rather than the study's total experimental coverage. Across method-development and validation slices, the experiments cover 275 unique usable Defects4J bugs, but these records do not all constitute independent held-out evidence. On the 38-bug main slice, selective reranking invokes the LLM for 11 cases and improves Top-1 from 0.3947 to 0.5526, Top-5 from 0.6053 to 0.7632, and MRR@10 from 0.4945 to 0.6513. A non-Closure Math 21–40 validation slice provides supporting evidence: the retrieval baseline is already strong, but selective reranking improves MRR@10 from 0.8167 to 0.8917 with 3 LLM calls. A later frozen follow-up applies the unchanged generic pipeline to 40 bugs from the previously unused Compress project. Its already strong retrieval baseline improves only at Top-1 and MRR@10, while the selector misses the remaining hard retrieval cases. This supports cross-project feasibility but not stable cross-project superiority. Real-world case studies on AboutWork and Easy Finance

test the feasibility of applying the same pattern beyond curated benchmark bugs. Additional full-rerank, selector-diagnostic, and repeated-run experiments explain the trade-off: sending every case to the LLM is more accurate, but selective invocation keeps cost visible and reveals where the selector misses hard cases. The claims remain limited to file-level ranking; this thesis does not claim automatic repair, line-level diagnosis, or full Defects4J-wide generalization.

The thesis makes three contributions. First, it provides a controlled empirical study of a non-oracle per-bug invocation gate within a retrieval-first, evidence-aware reranking pipeline. The gate sends selected bugs to one-shot LLM reranking while non-selected bugs retain deterministic retrieval fallback. Second, it quantifies how much rerank-all improvement selective invocation retains under reduced model requests, token usage, and runtime across controlled benchmark slices and repository case studies. Third, it analyzes the mechanisms that determine the trade-off, including candidate recall, evidence quality, selector false negatives, hosted-model variation, and controlled agentic/verifier ablations.

2 BACKGROUND

This section defines the fault-localization setting used in the thesis and explains the main technical concepts behind the retrieval–selector–rerank pipeline. The goal is to make the evaluation setting clear before the method and results sections introduce concrete datasets, prompts, and metrics.

2.1 Fault Localization

Fault localization techniques aim to reduce the search space a developer must inspect after a failure is observed. Classical spectrum-based fault localization uses information from passing and failing tests to assign suspiciousness scores to program elements [1, 2]. Broader surveys show that the field also includes mutation-based, model-based, statistical, and learning-based approaches [3, 4]. These techniques differ in the evidence they require, but they share a ranking goal: place likely faulty elements near the top so that developers inspect less irrelevant code.

Learning-based work expands the evidence available to localization models. DeepFL, for example, integrates multiple diagnostic dimensions, including suspiciousness, fault-proneness, and textual similarity [5]. Other recent work refines spectrum-based localization by filtering spectra with compiler-level type information [21]. These methods show that combining evidence sources can improve localization, but they also highlight a practical dependency: many techniques assume that test coverage, spectra, instrumentation, or structured diagnostic features are available. This assumption is not always satisfied for real bug records, especially when a report contains a user-facing symptom, a stack trace, or a failing test name but no complete coverage matrix.

This thesis focuses on file-level localization. The file-level setting is less precise than statement-level localization, but it has two practical advantages for the current work. First, it matches the granularity of many bug reports, stack traces, and fixing commits. Second, it can be evaluated across benchmark and real-world repositories without requiring uniform statement-level ground truth. The trade-off is that a Top-k hit only means that at least one modified file appears in the ranked list; it does not prove that all root-cause lines have been identified.

2.2 Retrieval-Based Candidate Generation

Retrieval-based bug localization treats a bug record as a query and source files as documents. The query can contain natural-language issue text, failing-test names, exception messages, stack traces, and other pre-fix evidence. A retrieval model then ranks files according to textual or structural relevance. BM25 is a standard lexical ranking model for this kind of first-stage search because it is deterministic, inexpensive, and scalable to many documents [6].

Software repositories add structure that ordinary text collections do not have. File paths, package names, class names, method names, identifiers, comments, and test names can all provide localization clues. IR-based bug localization systems therefore extend simple report-to-file matching with structured bug-report fields, source-code structure, change history, and stack-trace information [7–9, 22]. In this thesis, retrieval has a specific role: it constructs the candidate pool that later stages can rerank. Candidate recall therefore becomes an upper bound for LLM reranking. If the correct file is not retrieved, the LLM cannot rank it correctly without access to the whole repository.

2.3 LLM Reranking and Selective Invocation

Large language models are useful in this setting because bug localization evidence is heterogeneous. A model can compare a natural-language symptom with source-code snippets, stack traces, test names, and file metadata in a way that lexical retrieval cannot fully capture. Prior work on LLM-based localization and code representation shows that model behavior depends strongly on the code context, prompt design, diagnostic information, and representation granularity [10–15].

The thesis uses the LLM as a bounded reranker rather than as a whole-repository search engine. The retrieval stage first produces a candidate set. The evidence stage then extracts structured snippets and metadata for those candidates. The selector decides whether the case appears difficult enough to receive an LLM call. This design separates three sources of success or failure: whether retrieval includes the correct file, whether the evidence package exposes useful context, and whether the LLM ranks the candidate files well once it receives that evidence.

Selective invocation is central because LLM calls have token, runtime, and reproducibility costs. Calling a model for every bug may improve ranking accuracy under the current evidence package, but it also hides whether the extra cost is concentrated on cases that actually need semantic reranking. A non-oracle selector makes this trade-off explicit. It can save calls on easy cases, but false negatives leave some hard cases at the retrieval baseline. RQ2 therefore evaluates not only accuracy, but also candidate recall, evidence quality, selected fraction, token usage, runtime, and selected-versus-unselected behavior.

2.4 Agentic Inspection and Verifier Reranking

Agentic inspection extends one-shot reranking by giving an LLM a limited ability to inspect additional repository evidence before producing a final ranking. Related agentic software-engineering systems show that tool use can help models navigate repositories, execute tests, and reason over multi-file tasks [16–19, 23]. In this thesis, however, agentic inspection is not treated as the main method. It is an ablation that compares how extra tool-use steps change file-level ranking accuracy, model calls, runtime, and failure modes.

Verifier reranking is a second controlled extension. Instead of letting the first model prediction stand alone, a verifier-style pass reviews candidate rankings and evidence before the final output is evaluated. This can catch unsupported rankings in principle, but it also introduces another model call and another opportunity for evidence mismatch. RQ3 therefore compares one-shot reranking, agentic inspection, and verifier reranking in accuracy, cost, and failure modes under the same file-level localization boundary.

3 RELATED WORK AND STATE OF THE ART

[Ana review: completed] Related work is now organized through the SEGRES-style, Kitchenham-based synthesis used in the literature study, comparing method families by assumptions, evidence requirements, evaluation setting, and remaining gaps.

This section positions the thesis against information-retrieval-based bug localization, LLM-based fault localization, agentic software-engineering systems, and benchmark design. The organization follows the SEGRES-style, Kitchenham-based synthesis procedure used in the literature study: related work is not only summarized, but compared in terms of assumptions, evidence requirements, evaluation setting, and remaining research gaps [24]. These lines of work motivate the need for cost-controlled, file-level localization with structured evidence.

3.1 Information-Retrieval-Based Bug Localization

Information-retrieval-based bug localization treats a bug report as a query and source files as retrievable documents. BM25 is a standard probabilistic retrieval model for ranking text documents by query relevance [6]. In software repositories, the same idea can be applied to source files, identifiers, comments, stack traces, and structured bug-report fields. Early source-code retrieval work applied topic models such as latent Dirichlet allocation to bug localization [25]. BugLocator showed that retrieval over bug reports and source files can identify likely fixing locations, and later structured IR work improved localization by exploiting more explicit structure in bug reports and code [7, 8]. BLIA further combines bug reports with code-change history and stack-trace information [9]. Subsequent work also examined how bug-report content affects retrieval-based localization quality [22].

The development of IR-based localization is important for this thesis because it separates two ideas that are sometimes conflated in LLM-based systems: repository-scale search and semantic interpretation. Retrieval methods are effective at the first problem. They can index every source file, run quickly, and produce a reproducible ranking without model calls. They are weaker at the second problem when the bug report and faulty code use different vocabulary, when a stack frame points indirectly to the relevant file, or when the useful signal is a failing test rather than the issue description. This motivates using retrieval as the first stage rather than replacing it.

The proposed pipeline therefore keeps IR-based localization as the candidate-generation layer. This choice makes candidate recall a measurable upper bound for later LLM reranking: a reranker cannot recover a correct file that is missing from the candidate pool. It also creates a natural cost boundary. The system can run BM25 and focused hybrid retrieval for every bug, then invoke the LLM only when the selector predicts that lexical and deterministic signals may be insufficient. This is the core connection between the IR literature and RQ1/RQ2.

3.2 LLM-Based Fault Localization and Debugging

LLMs have recently been evaluated as fault-localization models and debugging assistants. A broader systematic review of LLMs for software engineering shows that LLM-based systems are now used across many software-development and software-maintenance tasks, while also raising recurring issues around datasets, input construction, prompting, and evaluation [26]. Fault-localization studies reflect the same pattern. Wu et al. study LLMs in fault localisation and show that results depend strongly on the provided code context, prompt design, and diagnostic information [10]. Yang et al. investigate test-free fault localization with language models, targeting line-level localization without relying on test coverage [11]. AutoFL studies explainable LLM-based localization, while other work studies the impact of fine-tuned code LLMs and LLM-driven iterative code exploration [12, 13, 27].

Several recent systems move closer to retrieval-augmented localization. SweRank frames issue localization as a code-ranking problem with a retrieval-and-reranking architecture [28]. BLAgent is especially close to this thesis because it studies file-level bug localization with agentic RAG, a compact retrieval-filtered candidate set, and evidence-grounded reranking [29]. FaR-Loc combines functionality extraction, semantic retrieval, and LLM re-ranking for fault localization [30]. SieveFL emphasizes hierarchical filtering and runtime-aware pruning before LLM-based localization [31]. RGFL uses reasoning-guided localization in the context of automated program repair [32]. These systems strengthen the state-of-the-art case for retrieval-augmented LLM localization. The remaining question for this thesis is narrower: whether a non-oracle per-bug invocation gate can retain useful reranking gain under constrained calls, tokens, and runtime in a consistent file-level protocol by allowing some bugs to bypass the LLM and keep deterministic retrieval fallback.

This line of work shows that LLMs can use natural-language and code context, but it also exposes a design problem. A model may need enough context to connect the report, failing behavior, and source code, yet repository-scale context is too large to include naively. Code representation work such as CodeBERT and UniXcoder addresses this issue from the representation side by learning code and natural-language representations that can support retrieval, ranking, or downstream software-engineering tasks

[14, 15]. Prompt-based LLM localization addresses it from the interaction side by selecting what code and diagnostic evidence the model sees.

Unlike direct whole-context localization approaches, this thesis narrows the repository before LLM use. However, recent systems such as SweRank, BLAgent, FaR-Loc, SieveFL, and RGFL also employ retrieval, filtering, or bounded candidate reasoning [28–32]. The distinction of this thesis is therefore not candidate narrowing itself, but the explicit per-bug routing decision: selected bugs receive one-shot reranking, whereas non-selected bugs bypass the LLM and retain deterministic retrieval fallback. This distinction matters for the evaluation: RQ1 asks whether selective use improves file-level rankings, while RQ2 asks whether candidate recall, snippet quality, and selective invocation explain the resulting accuracy–cost trade-off.

3.3 Adjacent LLM-Based Testing and Vulnerability Tasks

LLM-based software testing research has expanded beyond fault localization into flaky-test classification and repair, UI test repair, vulnerability detection, and test execution. Flaky-test work is relevant because it shows how difficult it is to reason from failure observations alone. Empirical studies and surveys document the causes and consequences of nondeterministic tests [33, 34]; datasets and predictors then attempt to detect flaky behavior without repeatedly rerunning every test [35–37]. Root-cause classification and repair-oriented work such as FlakyCat, LLM-based flaky-test classification, and FlakyFix show that model predictions depend on how failure evidence is represented and how repair categories are defined [38–40].

UI testing and test repair provide a second adjacent line. Record/replay failures often arise because UI attributes, application structure, or event sequences change over time [41]. Recent LLM-based UI test repair and general test-repair work studies how language models use failing tests and code context to propose or prioritize fixes [42, 43]. These studies are not file-level localization baselines, but they support the thesis assumption that failing-test context and source snippets are valuable evidence. They also warn that brittle context selection can lead to plausible but incorrect model behavior.

Vulnerability-detection work provides a third adjacent line because it studies LLM reasoning over code under realistic benchmark constraints. Recent studies show that vulnerability detection is sensitive to benchmark construction, inter-procedural context, data quality, and the difference between synthetic and real-world examples [44–48]. The same concern applies to fault localization: a model's ranking is meaningful only if the candidate files, evidence snippets, and evaluation labels reflect the debugging situation being claimed.

These adjacent tasks motivate the thesis design rather than serving as direct baselines. They show that LLM performance in software testing depends on task framing, dataset quality, available context, and validation protocol. This thesis therefore treats the LLM as a bounded reranker with explicit evidence and fallback, rather than as an unconstrained testing or repair agent. It also separates benchmark validation from real-world case-study evidence so that the evaluation does not overstate generality.

3.4 Agentic Software Engineering Systems

[Ana review: completed] Project Glasswing and Mythos Preview are now connected to RQ3 as practical evidence that tool-using, multi-step investigation is relevant, while their cybersecurity task boundary is kept separate from the thesis's file-level numeric evaluation.

Agentic software-engineering systems give an LLM tools for navigating repositories, editing files, running commands, or inspecting execution feedback. Feldt et al. frame conversational LLMs as autonomous testing agents, and ExecutionAgent studies LLM-based setup and test execution across arbitrary projects [18, 19]. SWE-bench frames real GitHub issues as repository-level repair tasks and shows that resolving issues often requires long-context reasoning across multiple files [23]. SWE-agent demonstrates that an agent-computer interface can improve an LLM agent's ability to navigate and edit repositories [17]. Agentless shows that a structured localization–repair–validation pipeline can compete with more complex software-engineering agents, while LocAgent uses graph-guided agents for code localization [49, 50]. In the fault-localization setting, AgentFL scales LLM-based localization toward project-level context by decomposing the task into comprehension, navigation, and confirmation steps [16].

These systems differ from one-shot reranking in both capability and risk. Tool use can expose evidence that was not present in the original prompt, which is useful when the first evidence package is incomplete. At the same time, each additional step can consume tokens, select irrelevant files, follow misleading search terms, or introduce non-deterministic behavior. Industry-facing systems such as Project Glasswing and Mythos Preview make this comparison practically relevant because they show that frontier-model developers are exploring agentic vulnerability discovery, where models use tools and multi-step investigation to search for security issues in realistic code settings [51, 52]. However, these reports target cybersecurity discovery rather than ordinary file-level fault localization, and they are public technical reports rather than peer-reviewed localization benchmarks. This thesis therefore uses them as qualitative context for RQ3, not as numeric baselines.

These systems motivate the RQ3 ablations in this thesis, but they are not the main method. The experiments compare controlled agentic inspection and verifier reranking against one-shot reranking while keeping the task at file-level localization. This separation keeps the main contribution focused on a cheaper selective reranking pipeline, while still testing how additional LLM interaction changes accuracy, cost, and failure modes on the available diagnostic slices.

3.5 Benchmark Choice: Defects4J and Alternatives

Defects4J is a widely used benchmark of real Java bugs with reproducible buggy and fixed versions, tests, and metadata [20]. It is suitable for controlled evaluation of Java debugging techniques and supports the frozen Closure and Math validation slices used in this thesis. Its main advantage is reproducibility: the same buggy

version, triggering tests, and fixing changes can be inspected repeatedly under a fixed protocol. This makes Defects4J appropriate for measuring Top-k, MRR@10, candidate recall, and reranking changes under controlled conditions.

The limitation is that Defects4J is not the same as an industrial stream of bug reports. Projects, languages, histories, build systems, and bug-record quality are constrained by the benchmark design. Real testing environments also include workflow, integration, configuration, and ecosystem-specific sources of failure, as shown by studies of CI/CD UI testing, microservice testing, and open-source IoT platforms [53–56]. These studies motivate using AboutWork and Easy Finance as case studies: they provide more realistic repository context, but their evidence is smaller and noisier than a curated benchmark.

SWE-bench provides a different view of software-engineering evaluation by asking models to resolve real GitHub issues through code changes [23]. That benchmark is closer to end-to-end repair, while this thesis evaluates file-level localization ranking. The distinction matters: a localization method can help direct developer or repair effort without generating a patch, but it should not be evaluated as if it solved full issue resolution. For this reason, the thesis uses Defects4J for controlled ranking evaluation and AboutWork/Easy Finance as real-world case studies, while explicitly limiting its claim to file-level localization.

3.6 Selected State-of-the-Art Comparison Targets

[Ana review: completed] The requested state-of-the-art comparison is now complete at the thesis's valid comparison boundary. Targets are selected from the related-work synthesis and assigned explicit roles: same-protocol numeric baselines, mechanism-level comparison, qualitative positioning, or controlled RQ3 ablations. Published scores from incompatible tasks are not presented as direct numeric baselines.

The preceding review applies this SEGRES-style, Kitchenham-based synthesis by grouping prior work into method families and comparing each family by evidence requirements, localization granularity, evaluation setting, and compatibility with the thesis protocol. This organization now determines the comparison strategy. IR-based bug localization defines the direct retrieval baseline family; spectrum-based and learning-based localization defines an important but evidence-incompatible comparison family; LLM-based localization and recent retrieve-and-rerank systems define the closest mechanism-level state of the art; and agentic software-engineering or vulnerability-discovery systems, including Project Glasswing and Mythos Preview, define the practical motivation for testing controlled tool use. This mapping is important because each family has a different comparison role: some can be evaluated numerically under the thesis protocol, while others support qualitative positioning, research-gap definition, or RQ3 ablations.

The thesis selects state-of-the-art comparison targets according to task compatibility rather than by copying published scores across incompatible settings. A fair numeric comparison requires the same input boundary, output granularity, no-leakage protocol, and evaluation labels. Table 1 therefore separates direct experimental baselines from qualitative comparison targets and controlled ablations.

This comparison strategy makes the empirical claim narrower but more defensible. The thesis compares numerically against baselines that can be run under the same file-level protocol and input boundary within each dataset. For Defects4J and AboutWork, this is a pre-fix evidence boundary; for Easy Finance, it is a retrospective commit-derived boundary. Other state-of-the-art systems are used to identify assumptions, missing comparisons, and ablation targets, but not as cross-paper score baselines.

3.7 State-of-the-Art Comparison and Research Gap Matrix

[Ana review: completed] The research-gap matrix now states what each method family establishes, what remains insufficiently evaluated, and which of RQ1–RQ3 addresses the resulting gap.

Table 2 consolidates the state of the art into the research gaps that support the three thesis research questions. The matrix is derived from the literature-study synthesis and the comparison boundaries in Table 1, rather than from cross-paper metric transfer: it identifies what existing work already establishes, what remains insufficiently evaluated, and how each gap is addressed by the thesis design.

The matrix leads to three thesis-level gaps. First, LLM-based fault localization needs evaluation that connects controlled benchmark slices with real repository case studies; this motivates RQ1. Second, recent retrieval-augmented systems show that LLM reranking can improve localization, but the accuracy–cost behavior of selective invocation depends on whether the correct file is retrieved, whether the evidence package exposes useful context, and whether a non-oracle selector routes hard cases to the model; this motivates RQ2. Third, agentic inspection and verifier reranking should be tested as controlled ablations rather than assumed to be better because they use more LLM interaction; this motivates RQ3.

Table 1: Selected state-of-the-art comparison targets and comparison type

Comparison target	Why it is relevant	How this thesis compares	Boundary
IR-based bug localization, including BugLocator, structured IR, and BLIA [7–9, 22]	These methods are the closest task family for ranking files from bug reports and repository text.	Direct numeric comparison through retrieval-only and focused hybrid retrieval baselines under the same file-level protocol.	The thesis does not reimplement every historical system; it compares against the retrieval family that shares the same input/output boundary.
Traditional and learning-based fault localization, including spectrum-based and feature-combination methods [1, 2, 5, 21]	These methods represent strong localization lines when coverage, spectra, or structured diagnostic features are available.	Qualitative comparison of evidence assumptions and task boundary.	They are not direct baselines here because the real-world records do not provide uniform spectra or statement-level coverage.
LLM-based fault localization, including test-free and explainable localization systems [10–13, 16, 27]	These systems are the closest LLM-based state of the art for using code and natural-language context.	Mechanism-level comparison against bounded one-shot reranking, full rerank-all diagnostics, and controlled agentic/verifier variants.	Published scores are not imported because granularity, datasets, candidate scope, and prompt/tool protocols differ.
Retrieve-and-rerank or pruning-based localization systems [28–32]	These works are closest to the thesis mechanism because they narrow code context before LLM-based ranking or reasoning.	Qualitative state-of-the-art comparison, plus internal full-rerank diagnostics that isolate the selector's effect under the thesis protocol.	Direct numeric comparison is left for future work because these systems differ in localization granularity, datasets, runtime evidence, repair integration, agentic protocol, or learned model components.
Repository-level agents, repair benchmarks, and vulnerability-discovery reports [17–19, 23, 49–52]	These systems test whether LLMs can use tools or repository context across multiple steps in repair, testing, localization, or security investigation.	Qualitative comparison plus RQ3 controlled ablations for agentic inspection and verifier reranking.	Their primary tasks include test execution, repair, vulnerability discovery, or project-level interaction rather than the file-level ranking task studied here.

Table 2: State-of-the-art comparison and research gap matrix

State-of-the-art line	What existing work establishes	Remaining gap for this thesis	Supported RQ
IR-based bug localization and BM25 baselines [6–9, 22, 25]	Retrieval methods provide reproducible and inexpensive file-ranking baselines from bug reports, source text, and structured repository information.	Lexical retrieval can miss semantic evidence from failing tests, stack traces, identifiers, and local snippets; it also does not decide when expensive reranking is worthwhile.	RQ1, RQ2
Traditional and learning-based fault localization [1–5, 21]	Spectrum-based and learning-based methods show that diagnostic signals can improve localization when coverage, spectra, or structured fault features are available.	Many practical bug records do not provide uniform coverage or statement-level spectra, so the thesis focuses on file-level localization from structured repository evidence while separating pre-fix and retrospective case-study inputs.	RQ1
LLM-based and retrieval-augmented fault localization [10–15, 27–32]	Recent work shows that LLMs can use code and natural-language context for fault localization, and that retrieval, filtering, reasoning, and prompt design strongly affect performance.	Existing systems show the promise of retrieval-augmented LLM localization, but they do not isolate the accuracy–cost behavior of a non-oracle per-instance invocation gate under a consistent file-level benchmark and case-study protocol.	RQ1, RQ2
Adjacent LLM-based testing and vulnerability tasks [37, 38, 40–42, 44–46]	LLMs and code models have been evaluated on flaky-test prediction, UI test repair, vulnerability detection, and other testing tasks where context and benchmark design affect results.	These tasks are related but not direct file-level localization baselines; they mainly motivate evidence control, benchmark caution, and explicit failure analysis.	RQ1, RQ2
Agentic software-engineering and vulnerability-discovery systems [16–19, 23, 49–52]	Agentic systems show that LLMs can navigate repositories, use tools, and perform multi-step investigation in repair, project-level localization, test execution, or vulnerability-discovery settings.	It remains unclear how controlled agentic inspection and independent verifier reranking compare with one-shot reranking in accuracy, cost, and failure modes for ordinary file-level localization.	RQ3
Benchmark and real-repository evaluation [20, 23, 47, 53–55]	Defects4J supports controlled Java bug evaluation, while repository-level benchmarks and case studies expose more realistic project context.	Benchmark evidence and real-world case-study evidence have different strengths. The thesis therefore separates controlled Defects4J validation from AboutWork and Easy Finance case-study evidence.	RQ1

4 DESIGN

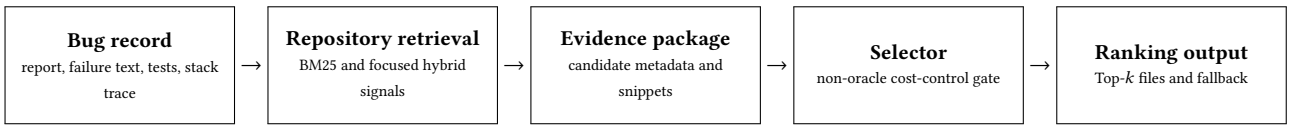
The method is a retrieval-first pipeline with selective LLM reranking. Retrieval scans the repository and produces a bounded candidate pool. Evidence construction turns the bug context and candidate files into compact, structured inputs. A non-oracle selector then decides whether the case appears difficult enough for LLM reranking. Selected cases are reranked by a one-shot DeepSeek prompt, while non-selected cases keep the retrieval ranking through fallback.

This design separates repository search from semantic reranking. Repository search remains deterministic and inexpensive. The LLM is used only after the search space has been narrowed. Each reranking decision must be explainable in terms of bug context, candidate metadata, snippets, and retrieval evidence available before the fix. Figure 1 summarizes the pipeline and the selector branch.

4.1 Problem Definition

Let b denote a bug record and let S_b denote the corresponding buggy repository snapshot. The repository snapshot is indexed as a set of candidate source files $C_b = \{c_1, c_2, \ldots, c_n\}$. Each candidate file is represented by its path, package or module context when available, class and method names, and source text. Test files, generated files, build artifacts, dependency directories, and other non-target files are excluded by the repository indexer.

A localization method produces a ranking R_b, which is an ordered list of files from C_b. For retrieval-only baselines, R_b is the BM25 or focused hybrid retrieval order. For selected LLM cases,



Selected branch	The bug is sent to one-shot reranking. The model can reorder only retrieved candidates, and invalid or missing entries are completed with retrieval fallback.
Fallback branch	The bug bypasses the model and keeps the focused retrieval ranking. This branch keeps every case evaluable and makes LLM cost explicit.
Evaluation	Top-k, MRR@10, candidate recall, selected fraction, tokens, runtime, and failure categories are computed after prediction, using repair-derived labels only after prediction and respecting the Easy Finance retrospective query boundary described in Section 5.

Figure 1: Selective evidence-aware fault-localization pipeline. The method separates repository-scale retrieval from bounded LLM reranking and keeps repair diffs, changed-file labels, and post-fix source code outside all prediction steps; Easy Finance uses commit-message-derived query text under a retrospective boundary.

R_b is the normalized and merged output after reranking and fallback. Let G_b denote the ground-truth source-file set derived from the fixing evidence. G_b is hidden from the method and used only when computing Top-k, MRR@10, and candidate recall.

The method is constrained by candidate recall. If no file in G_b appears in the retrieved candidate pool, the LLM cannot recover the correct file because it is only allowed to rerank existing candidates. Candidate recall is therefore reported separately from final Top-k accuracy.

4.2 Focused Hybrid Retrieval

The first stage builds a source-file index for the buggy snapshot. Each indexed document combines the file path, package or module declaration, class names, method names, and file content. BM25 ranks files against a bug query built from the bug report, failure text, triggering tests, and filtered stack-trace information where available.

Focused hybrid retrieval augments this lexical ranking with deterministic debugging signals. These signals include stack-trace class and file matches, triggering-test matches, exception type and method names, identifier overlap, direct file or path hints, and framework-specific evidence when the bug context justifies it. The goal is not to replace BM25 with hand-written rules, but to prevent high-signal debugging evidence from being diluted by long stack traces, framework frames, or unrelated report text.

The retrieval stage returns a bounded candidate pool for downstream reranking. In the main reranking configuration, the pool contains up to `top_candidates` = 50 files. This size is a practical compromise: it is large enough to preserve recall in many cases, but small enough for evidence construction and LLM prompting to remain tractable.

4.3 Evidence Construction

Evidence construction turns retrieval output into a compact package for reranking. Bug-level evidence includes the issue or bug text, failure message, triggering tests, filtered stack trace, exception information, and any available reproduction context. Candidate-level evidence includes file paths, class and method names, retrieval scores, deterministic match reasons, and selected source snippets.

Snippet selection is important because the LLM does not receive entire repositories. The implementation prefers high-signal locations such as stack frames, query-term matches, failing-output patterns, assertion-related code, and local source regions around relevant identifiers. It also avoids repeatedly spending context on low-value framework frames or large generic utility sections when the bug evidence points elsewhere. Retrieval evidence is treated as weak support rather than ground truth: a high retrieval score can justify attention, but it cannot by itself prove that a file is faulty.

The evidence package is designed to expose why a candidate is plausible. A file may be plausible because its class name appears in the stack trace, because a triggering test names a related component, because a source snippet contains the failing state transition, or because identifiers in the bug report overlap with methods in the file. If the relevant code does not appear in the retrieved pool or selected snippets, reranking is expected to fail; these cases are analyzed as retrieval or evidence-construction failures rather than as pure LLM failures.

4.4 Non-Oracle Selective Rerank Gate

The selector decides whether to call the LLM for a bug. It is non-oracle: it cannot inspect ground-truth files, repair diffs, or post-fix code. It only uses retrieval scores, candidate metadata, bug text, stack-trace structure, snippet signals, and deterministic evidence from the input record. For Defects4J and AboutWork, this input is treated as pre-fix evidence. For Easy Finance, the input bug text is commit-derived retrospective metadata, as described in Section 5.

The selector is motivated by cost control. Many bugs are already localized well by retrieval, so reranking every case spends tokens and may introduce unnecessary instability. Other cases show signs that retrieval may be unreliable: a weak gap between top candidates, missing direct hints for the top-ranked file, multiple competing class or path hints, stack-trace ambiguity, or domain-specific patterns that lexical retrieval may misread. These signals route difficult-looking cases to the LLM while leaving easier cases on the retrieval fallback path.

The selector is not expected to be perfect. A false positive causes an unnecessary LLM call and can destabilize an already useful retrieval ranking if reranking moves a correct top file downward. A false negative leaves a difficult bug on the retrieval ranking and can reduce accuracy. RQ2 therefore evaluates not only final accuracy, but also selected fraction, token usage, runtime, selected-versus-unselected behavior, and rerank-induced degradations.

The selector is implemented as deterministic rules rather than as a trained classifier. Table 3 summarizes the selector variants used in the reported experiments. The rules are non-oracle because they inspect only the input bug text, retrieval scores, stack or test hints, candidate paths, and source-snippet signals, not labels or repairs. The exact selected bug IDs are saved in the selector JSON artifacts used by the reranking and merge scripts.

Table 4 gives the main thresholds and deterministic conditions needed to reproduce the reported selector behavior at the thesis level. The full branch logic and selected bug identifiers remain in the saved selector artifacts and in `scripts/select_rerank_candidates.py`.

At a high level, the selector follows the rule order below. Dataset-specific pattern checks instantiate the same rule family with project-specific terms recorded in `scripts/select_rerank_candidates.py`.

```
if direct high-confidence evidence supports the top retrieval file:
    keep retrieval fallback
elif top-candidate score gap is weak:
    select for LLM reranking
elif stack trace, test name, or path hints point to competing files:
    select for LLM reranking
elif project-specific failure pattern is detected:
    select for LLM reranking
else:
    keep retrieval fallback
```

4.5 One-Shot DeepSeek Reranking and Fallback

Selected cases are sent to a one-shot DeepSeek reranker. The prompt contains the bug evidence, the bounded candidate list, candidate snippets, and explicit rules that the model must use only the provided input information, not repair labels, patches, or post-fix code. The model is asked to return strict JSON containing the bug id and a ranked list of candidate files, each with a confidence score and a short reason.

The reported reranking experiments use the DeepSeek API model alias `deepseek-v4-flash`; the API documentation citation was accessed on 25 June 2026 [57], while individual API run dates are recorded in the experiment manifests. The rerank client sends one non-streaming chat-completion request per selected bug. It sets `temperature=0.0` and `response_format={``type'': ``json_object''}`. It does not explicitly set `top_p`, `max_tokens`, `thinking`, or `reasoning_effort`. DeepSeek documentation states that `deepseek-v4-flash` supports thinking and non-thinking modes, with thinking mode as the default; in thinking mode, `temperature` and `top_p` are accepted for compatibility but do not affect generation. The saved prediction files include provider-reported reasoning-token counts for many responses but do not retain `reasoning_content` or an explicit `thinking` field for every run. The effective provider-default reasoning mode is therefore treated as a reproducibility limitation rather than as a fully pinned model setting.

The frozen Closure, Math, and Compress validation protocols use `top_candidates=50`, `top_output=10`, and `max_snippet_lines=12`. Retrieval evidence and triggering-test context are enabled, with a 12,000-character limit for triggering-test context.

After the model returns, the output is normalized before evaluation. Files not present in the candidate pool are discarded. Duplicate file entries are collapsed. Malformed or missing fields are handled conservatively. The final ranked output keeps the model's valid ordering first and then fills any remaining positions from the original retrieval ranking. This fallback is necessary because an LLM may omit plausible candidates, return fewer than the requested files, or produce invalid entries.

Non-selected bugs bypass the LLM and keep the focused retrieval ranking. This means the selective method always has a defined output: selected cases use normalized reranking plus retrieval fallback, while non-selected cases use retrieval-only fallback. The comparison with full reranking and retrieval baselines makes it possible to separate the value of reranking from the value of selective invocation.

4.6 Controlled Agentic Inspection and Verifier Reranking

Controlled agentic inspection and verifier reranking are evaluated as RQ3 extensions and ablations, not as the main method. Agentic inspection lets an LLM inspect additional repository evidence within a fixed tool budget and candidate scope. The goal is to test whether limited tool use can improve difficult cases beyond the one-shot evidence package.

The agentic setting is deliberately constrained. It cannot access fixing commits, ground-truth files, repair patches, or post-fix source. Tool observations are recorded so that each decision can be audited. The fixed budget prevents the comparison from becoming an unconstrained search process where additional calls and inspection time are hidden from the cost analysis.

Verifier reranking is treated as a diagnostic ablation. Its purpose is to test whether a second LLM-style review of candidate rankings improves or destabilizes the result. The current thesis treats verifier behavior cautiously and reports it separately from the primary selective reranking pipeline.

4.7 No-Leakage Design

The no-leakage design defines what information each component may use. Retrieval may use the buggy source snapshot and bug-context artifacts such as reports, failing tests, stack traces, and reproduction text. Evidence construction may extract candidate metadata and source snippets from the buggy snapshot. The selector may use retrieval scores, deterministic match reasons, and the input bug evidence defined for the dataset. The one-shot reranker and agentic variants may inspect only the evidence and source context made available under these same constraints.

For Defects4J and AboutWork, fixing-commit information is reserved for evaluation. Easy Finance is the sole query-provenance exception: its natural-language query is retrospectively derived from the fixing-commit message. Across all datasets, repair diffs,

Table 3: Selector configurations used in the reported experiments

Selector	Dataset / split	Main input signals	Freeze status	Artifact
Closure cost-control v3	Closure 61–100	Score-margin and rank uncertainty; direct stack/test hints; pass-chain, type-cycle, code-output, validator, and state-reset patterns.	Frozen before the reported held-out reranking and follow-up diagnostics.	`outputs/closure_heldout_*_selector_closure_cost_control_v3.json`
Generic Math selector	Math 21–40	Top-candidate uncertainty, direct hints, and state-reset or related pattern signals.	Written before the Math fresh-validation run.	`outputs/math_fresh_21_40_selector_generic_t102_h7_patterns.json`
Frozen generic selector	Compress 1–40	Generic top-candidate uncertainty, direct hints, and existing non-project-specific pattern signals.	Frozen before constructing this previously unused project slice; no Compress-specific rule was added.	`outputs/compress_frozen_1_40_selector_generic_t102_h7_patterns.json`
AboutWork selector_v3	AboutWork committed-60	Production BM25 rank evidence, low-ratio or ambiguous top candidates, and bug-log/source-snippet evidence.	Case-study/method-development selector used for the saved AboutWork run; not claimed as a general industrial selector.	`outputs/aboutwork60_selector_v3.json`
Easy Finance selector_v1	Easy Finance clean63 / strict62	Production BM25 rank evidence plus UI-specific evidence signals from commit-derived bug text, candidate paths, and snippets.	Case-study/method-development selector used for the saved Easy Finance runs; not an independent transfer validation.	`outputs/easy_finance_*_prod_selector_v1.json`
Mockito cost-control variants	Mockito diagnostic slices	Direct test/class hints, rank uncertainty, and Mockito-specific pattern terms.	Diagnostic/fresh-validation use only, not part of the main held-out claim.	`outputs/mockito_*_selector*.json`

Table 4: Main selector rule conditions in the reported experiments

Rule family	Reported condition	Effect
Score-ratio uncertainty	Closure cost-control v3 and the generic Math and Compress selectors select when the top-two retrieval score ratio is at most `1.02`, subject to their frozen filters. AboutWork selector_v3 also uses `1.02`.	Routes close retrieval competitions to reranking.
Top-1 lacks direct hint	Closure, Math, and Compress enable `include_top1_without_direct`; the reported selector criteria do not impose a minimum or maximum direct-hint rank for this rule.	Routes cases where the top retrieval file lacks direct stack, test, path, or class support.
Many direct hints	Closure, Math, and Compress use `direct_hint_count_threshold=7`.	Routes cases with many competing direct class or path hints.
Project-pattern evidence	Closure cost-control v3 enables deterministic pass-chain, type-cycle, code-output, validator, and state-reset pattern filters. The saved criteria set `pass_chain_min_boost=1000.0`; the script records the remaining pattern thresholds and rank checks.	Routes selected project-specific failure patterns while filtering some low-value false positives.
Easy Finance case-study rules	Easy Finance selector_v1 disables score-ratio selection (`score_ratio_enabled=false`) and uses named backend/frontend mismatch flags, including bookkeeping, reports, admin invoice, management-command, unreviewed-table, admin-user, chat-history, loading, expense-form, and invoice-currency signals.	Routes retrospective case-study records selected by deterministic domain-pattern rules.

changed-file labels, fixed or post-fix source code, and post-hoc knowledge of the modified files remain excluded from retrieval, selector decisions, LLM reranking, and agentic inspection. These artifacts are loaded only by evaluation scripts after predictions have been produced.

The implementation also uses output validation to reduce leakage-like failures from model behavior. The LLM is instructed to rank only files from the candidate set, and normalization removes files outside that set. This does not make the model infallible, but it keeps the evaluated output within the candidate universe produced by retrieval.

5 EVALUATION SETUP

5.1 Evaluation Overview

The experiments follow the three research questions introduced in Section 1. Table 5 maps each question to the corresponding experiment and metric family.

The evaluation is organized in four layers. First, retrieval-only baselines and selective one-shot reranking are compared under the same file-level protocol to answer RQ1. Second, candidate recall, evidence-package diagnostics, full-rerank comparisons, and selected-versus-unselected behavior are used to explain the accuracy–cost mechanisms in RQ2. Third, controlled agentic inspection and verifier reranking are compared with one-shot reranking for RQ3. Finally, the analysis applies a common measurement boundary across these layers: ranking effectiveness is reported with Top-k and MRR@10, resource use with model requests, tokens, and wall-clock runtime, and prediction inputs are separated from repair-derived evaluation labels according to the no-leakage protocol described below.

5.2 Method Boundary

The main method evaluated in this thesis is selective one-shot reranking. The pipeline is:

```
bug context
-> focused hybrid retrieval
-> deterministic evidence construction
-> selective rerank gate
-> one-shot DeepSeek rerank for selected cases
-> retrieval fallback for non-selected cases
-> file-level evaluation
```

Controlled agentic inspection and verifier reranking are not part of the main method. They are evaluated separately as RQ3 extensions and ablations. This separation is important because additional LLM steps may increase model calls, token usage, and runtime without consistently improving ranking quality.

Table 5: Mapping between research questions, experiments, and metrics

RQ	Experiment / Analysis	Main Metrics
RQ1	Compare retrieval-only baselines and selective evidence-aware LLM reranking on Defects4J validation slices and real-world repository case studies.	Top-1, Top-3, Top-5, Top-10, MRR@10, raw hit counts.
RQ2	Analyze candidate recall, evidence/snippet quality, selector coverage, selected-versus-unselected behavior, token usage, runtime, and cost-control outcomes.	Recall@K, selected fraction, model requests, tokens, runtime, MRR@10 change, failure categories.
RQ3	Compare one-shot reranking with controlled agentic inspection and verifier reranking.	Top-k, MRR@10, model calls, tokens, runtime, per-bug rank changes, failure modes.

5.3 Comparison Strategy

The direct experimental comparisons use methods that can be run under the same file-level, no-leakage protocol. Retrieval-only and focused hybrid retrieval represent the information-retrieval bug-localization family discussed in Section 3. Selective one-shot reranking is the proposed budgeted method. Full one-shot rerank-all is included as a diagnostic comparison that removes the selector and sends every case to the same reranker under the same evidence package. Controlled agentic inspection and verifier reranking are evaluated as internal extensions under the same candidate and evaluation boundary.

Other state-of-the-art systems are compared qualitatively rather than by importing their published scores. Spectrum-based and learning-based approaches often require coverage, spectra, or statement-level diagnostic features that are not uniformly available for the real-world case-study records. Existing LLM fault-localization and repository-agent systems differ in granularity, candidate scope, tool protocol, or end task. Cross-paper score comparison would therefore mix different labels and assumptions. This thesis uses those systems to define the research gaps and comparison targets, while keeping the numeric results restricted to methods evaluated on the same records and file-level metrics.

5.4 Datasets

Table 6 summarizes the datasets used in the experiments. Defects4J pilot data is used for method development and diagnostic analysis. Closure 61–100 frozen held-out validation is the main benchmark result. Math 21–40 provides non-Closure fresh validation, and Compress 1–40 is a later frozen cross-project follow-up on a previously unused project. AboutWork and Easy Finance are used as real-world repository case studies.

Closure 21–60 and the Mockito fresh-validation rows document development and diagnostic coverage rather than the main held-out conclusion. The main Results claims use Closure 61–100 as the frozen held-out aggregate, with Math 21–40 as supporting non-Closure validation. Compress 1–40 is reported separately as an August 2026 follow-up because it was run after the original validation campaign, using a project that had not contributed to selector, prompt, evidence-rule, or error-analysis development. Closure 61–100 contains 38 usable records because `Closure-63` and `Closure-93` are not listed in the active-bugs metadata for the current Defects4J checkout and were skipped during dataset construction. The development and validation slices contain 275 unique usable Defects4J bugs: 120 pilot bugs, 40 Closure 21–60 bugs, 38 Closure 61–100 bugs, 20 Math 21–40 bugs, 17 Mockito fresh-validation bugs, and 40 Compress follow-up bugs. The 10-case RQ3 diagnostic mini-benchmark reuses bugs from these slices and is not counted as an additional unique set. Mockito is not used for an additional held-out slice because the active bugs available in the current Defects4J checkout are only 1–38, and these have already been covered by the pilot and fresh-validation experiments.

5.5 Real-World Case-Study Records

AboutWork and Easy Finance are included to test whether the same file-level localization pipeline can be applied outside curated benchmark bugs. They are not treated as statistically representative industrial datasets. Their role is to expose the method to real repository vocabulary, frontend/backend code organization, business-domain issue descriptions, noisy bug text, and imperfect fixing labels.

AboutWork committed-60 is built from a company bug log and committed fixes. Each record contains a bug id, bug text, repository side, buggy commit, fixed commit, source directory, pre-fix failure or reproduction text when available, and ground-truth files derived from the fixing evidence. The records include backend Python files and frontend TypeScript/TSX files. Several records contain business-specific chatbot, admin, HR, scheduling, document, authentication, and UI behavior descriptions, which makes them different from the Java library bugs in Defects4J. The dataset builder records excluded entries and path remappings, and each JSONL record marks fix metadata and ground-truth files as evaluation-only information.

Easy Finance clean63 is a retrospective git-history-derived case study. Candidate commits were selected from backend and frontend repositories when the commit message described a bug, crash, wrong behavior, route issue, loading issue, or similar failure, and when the fix touched source files rather than only dependencies, build files, secrets, migrations, or deployment metadata. The bug text in these records is inferred from fixing-commit metadata, recorded as `fixed_commit_message_seed` in the JSONL artifacts. Therefore, Easy Finance evaluates retrospective file ranking from commit-derived bug descriptions rather than a fully prospective pre-fix localization scenario. The fixing commit, repair diff, post-fix source code, and ground-truth

Table 6: Experimental datasets

Dataset	Role	Size	Thesis Use
Defects4J pilot	Method development and pilot validation	120 bugs	Pilot result
Closure 21–60	Fresh validation	40 bugs	Selector/rule validation
Closure 61–100	Frozen held-out aggregate	38 bugs	Main benchmark result
Math 21–40	Non-Closure fresh validation	20 bugs	Supporting Defects4J validation
Compress 1–40	Frozen cross-project follow-up	40 bugs	Independent-project RQ1/RQ2 support
Mockito 21–30	Fresh attempt	9 usable bugs	Diagnostics / fresh validation
Mockito 31–38	Fresh validation	8 bugs	Fresh validation
AboutWork committed-60	Real bug-log case study	60 records	RQ1; RQ3 extension
Easy Finance clean63	Retrospective git-history case study	63 records	RQ1
Easy Finance strict62	Filtered retrospective git-history case study	62 records	RQ3
Defects4J diagnostic mini	Diagnostic extension benchmark	10 bugs	RQ3

files are still hidden from retrieval, selector decisions, and LLM prompts, and are used only for evaluation. The records include both backend Python files and frontend TypeScript/TSX files. Easy Finance strict62 is a filtered sensitivity set that removes one record, `easyfinance-frontend-20250930-001`, whose bug text and touched files were judged insufficiently aligned for the stricter RQ3 comparison. The clean63 set is used for the main RQ1 case-study result, while strict62 is used for agentic/verifier analysis.

Because Easy Finance queries are commit-derived, a direct-target-hint audit checks whether the query text discloses ground-truth file identifiers. Table 7 reports the audit. No clean63 record contains an exact ground-truth path or filename with extension. Four records contain an exact component-stem phrase, such as a page or table component name, and 42 records contain broader component-level hints. These records are retained in the reported retrospective case study, so the Easy Finance results should be read as a commit-history localization sensitivity setting rather than as a fully prospective issue-report dataset.

Table 7: Easy Finance direct-target-hint audit

Dataset	N	Exact path/file	Exact comp.	Comp. hint	No target id.	Removed
clean63	63	0	4	42	17	0
strict62	62	0	4	41	17	0

Exact path/file means a full ground-truth path or filename with extension appears in the query. Exact comp. means the full ground-truth file stem appears as words without the extension. Comp. hint means the query contains broader module or component words that overlap with the ground-truth path. The audit artifacts are stored in `docs/easy_finance_direct_hint_audit_2026-07-20.md`.

5.6 Ground-Truth Labels

Ground truth is used only after prediction for evaluation. For Defects4J, the ground-truth file set is derived from source files modified by the official fixing change. For the real-world case studies, the same file-level principle is applied to the available issue, commit, and bug-log evidence. These labels can be noisier because a fixing commit may combine localization-relevant edits with cleanup, refactoring, formatting, or nearby changes.

The evaluation therefore asks whether a ranking places at least one relevant modified file early in the list. It does not assume that every edited file is a root cause, and it does not measure whether all modified files are recovered. The exact any-hit Top-k and MRR@10 definitions are given in the Evaluation Metrics subsection.

5.7 No-Leakage Protocol

The experiments separate input information from evaluation information. Information allowed in retrieval, selector decisions, prompts, and agent observations includes bug reports or issue text, failing tests, triggering tests, stack traces, buggy source files, and deterministic candidate evidence. For Defects4J and AboutWork, fixing-commit information is used only for evaluation.

For Easy Finance, this no-leakage boundary has a narrower interpretation. The model still does not receive the full fixing commit, repair diff, post-fix code, or ground-truth file labels, but the natural-language bug text is retrospectively derived from commit metadata rather than from an independently recorded pre-fix issue. Across all datasets, repair diffs, fixed or post-fix source code, changed-file labels, and post-hoc knowledge of the modified files are excluded from prediction components. The Easy Finance results are therefore reported as retrospective case-study evidence, not as proof that the same inputs would have been available before the fix.

Closure 61–80 and 81–100 were evaluated after writing frozen protocols and before running dataset build, retrieval, selector, reranking, and evaluation. Error analysis from the first held-out slice was not used to modify the protocol for the second slice. Math 21–40 similarly used a written protocol before dataset build, retrieval, selector, DeepSeek reranking, and evaluation. Compress 1–40 used a separate protocol written before dataset construction and retained the existing generic selector without Compress-specific rules. A network read timeout interrupted the selected-case API batch after its first completed result; that result was preserved, and the five remaining preselected cases were rerun with only the client timeout increased from 120 to 300 seconds.

5.8 Ethics, Confidentiality, and Data Handling

The real-world case studies use private project records and therefore have a different reproducibility boundary from Defects4J. The public thesis reports aggregate metrics, selected/unselected behavior, and artifact paths, but it does not reproduce full private repositories, full raw company logs, credentials, secrets, internal URLs, customer data, or complete prompt packages. Prompt construction uses bounded source snippets and candidate metadata rather than whole repositories.

The hosted reranking experiments transmitted bounded bug text, candidate metadata, and selected source snippets to the external model provider. The use of bounded private bug records and source snippets with the hosted model was approved by the relevant project owner or organizational contact. Before use in prompts, the private case-study inputs were bounded to selected snippets and screened to avoid credentials, secrets, personal information, customer data, and internal URLs. The experiment manifests therefore record the provider, model alias, access date, prompt/evidence directory, output checksum, token usage, and runtime for the reported DeepSeek runs. Public reproducibility is strongest for Defects4J; for the private case studies, reproducibility is limited to the documented schema, scripts, aggregate metrics, and non-sensitive artifact descriptions unless project-specific sharing approval is available. Releasing complete private prompts, raw logs, or source snippets would require project-owner approval and an additional screening pass for credentials, secrets, personal information, customer data, and internal URLs.

5.9 Evaluation Metrics

The task is evaluated as file-level ranking. For a bug b, let G_b be the set of ground-truth modified files and let $R_b[1:k]$ be the first k files returned by a method. Top-k accuracy is:

$$Top\text{-}k(b) = \begin{cases} 1, & \text{if } G_b \cap R_b[1:k] \neq \emptyset \\ 0, & \text{otherwise.} \end{cases}$$

The final Top-k score is the average across all evaluated bugs. This thesis reports Top-1, Top-3, Top-5, and Top-10.

Mean Reciprocal Rank at depth 10 (MRR@10) measures how early the first correct file appears within the same Top-10 output boundary used by the final merged predictions:

$$\text{MRR@10} = \frac{1}{|B|} \sum_{b \in B} RR_b^{10},$$

where $RR_b^{10} = 1/rank_b^{10}$ if the first ground-truth file appears within the first 10 ranked files, and $RR_b^{10} = 0$ otherwise. This keeps retrieval baselines, selective merged outputs, full rerank-all outputs, and agentic variants on the same ranking depth.

For multi-file bugs, the evaluation uses an any-hit file-level definition: if any ground-truth modified file appears in the top-k results, the bug is counted as successful. This definition measures whether the system can guide a developer to at least one relevant

file, but it does not measure whether all modified files are identified.

Candidate recall is reported separately because LLM reranking can only reorder files that appear in the candidate pool. For selective reranking, the merged output is truncated to Top-10. Therefore, Top-20, Top-50, and Top-100 candidate recall are computed from the retrieval output, not from the merged Top-10 output or the MRR@10 calculation.

In the cost analysis, cost refers to provider/model requests, token consumption, and wall-clock runtime. Monetary API cost is not used as a primary metric because hosted-model pricing and alias mappings can change over time. Token counts are provider-reported totals; prompt, cache, and completion-token fields are retained in the output artifacts when the provider returns them.

5.10 Statistical Reporting

Top-k results are reported as both raw counts and percentages, because several evaluation sets are small and percentages alone may exaggerate differences. Per-bug quantities such as reciprocal rank, first-correct-file rank, token usage, and runtime are reported with mean, standard deviation, and median where applicable. The statistical supplement also reports McNemar exact tests for hit/miss Top-k outcomes, Wilcoxon signed-rank tests for paired per-bug reciprocal-rank-at-10 changes, Recall@K curves, and selected-versus-unselected subset metrics. Because the evaluation subsets are limited in size, p-values are used as descriptive supporting evidence rather than as the only basis for conclusions.

6 RESULTS

Following the comparison strategy in Section 5, the numeric results compare methods that run on the same records under the same file-level, no-leakage protocol. Published state-of-the-art systems from Section 3 are used as task and mechanism comparison targets, but their reported scores are not mixed with these results because their datasets, granularity, and tool protocols differ.

6.1 RQ1: Effectiveness Across Benchmark and Real-World Repository Data

6.1.1 Benchmark Validation on Defects4J. Table 8 reports the development/pilot results across six Defects4J projects. The pilot indicates that LLM reranking can help as a second-stage file-level reranker when the candidate pool already contains the correct file. Because these runs include targeted evidence add-ons, they are treated as method-development evidence rather than held-out generalization evidence.

The main held-out benchmark evidence comes from Closure 61–100. As shown in Table 9, selective DeepSeek reranking invokes the LLM for 11 out of 38 bugs and improves Top-1 from 0.3947 to 0.5526, Top-5 from 0.6053 to 0.7632, and MRR@10 from 0.4945 to 0.6513.

To reduce the risk that the main held-out evidence is specific to Closure, Math 21–40 is included as non-Closure fresh validation. This slice does not replace the Closure main benchmark; it is used as supporting validation. Table 10 shows that the retrieval baseline is already strong on Math, but selective reranking still improves MRR@10 from 0.8167 to 0.8917 while invoking DeepSeek for only 3 of 20 bugs.

Math 21–40 has Top-50 candidate recall of 1.0000. Selective DeepSeek moves `Math-31` from rank 26 to rank 1 and `Math-29` from rank 2 to rank 1. This provides supporting evidence beyond Closure, while also showing that Math 21–40 is easier than the Closure held-out slice.

A later frozen follow-up evaluates Compress 1–40, a Defects4J project that was not used in prior method development. The dataset, generic selector parameters, candidate-pool size, evidence settings, and stopping rule were fixed before construction and model calls. Table 11 shows a strong retrieval baseline: selective reranking raises Top-1 from 0.7500 to 0.7750 and MRR@10 from 0.8299 to 0.8465, while Top-3, Top-5, and Top-10 remain unchanged. The change is concentrated in `Compress-2`, whose first correct file moves from rank 3 to rank 1; the other five selected cases retain rank 1. This is evidence that the frozen pipeline can execute on a previously unused project, but it is not evidence of a stable aggregate improvement.

6.1.2 Real-World Repository Case Studies. Table 12 summarizes the real-world case-study results. AboutWork uses manually recorded company bug logs. Easy Finance uses git-history-derived records built from fixing commits.

Table 13 further breaks down the Easy Finance clean63 result by the query-hint categories from the direct-target audit. This analysis uses the same BM25 and selective UI-evidence-v2 prediction files as Table 12; it does not rerun retrieval or LLM calls. The categories are mutually exclusive and exhaustive, using the precedence

Table 8: Development / pilot results after method refinement

Project	Development Setting	Bugs	Top-1	Top-3	Top-5	MRR@10
Lang	BM25 + DeepSeek rerank	20	0.95	1.00	1.00	0.9750
Math	Hybrid/direct + compact evidence DeepSeek	20	0.90	0.95	1.00	0.9350
Chart	Focused hybrid/direct	20	0.85	0.95	1.00	0.9040
Time	Focused hybrid/direct + DeepSeek hard4	20	0.90	1.00	1.00	0.9500
Closure	Focused hybrid/direct + DeepSeek hard8 + pass-chain/type-cycle rules	20	0.40	0.80	1.00	0.6225
Mockito	Focused hybrid/direct + tight selector, 9 DeepSeek calls	20	0.65	1.00	1.00	0.8000

Table 9: Closure 61–100 frozen held-out aggregate

Dataset	Setting	Bugs	LLM Calls	Top-1	Top-3	Top-5	Top-10	MRR@10
Closure 61–100	Frozen retrieval baseline	38	0	0.3947	0.5263	0.6053	0.7105	0.4945
Closure 61–100	Frozen cost-control v3 + DeepSeek	38	11	0.5526	0.7105	0.7632	0.8421	0.6513

Table 10: Math 21–40 non-Closure fresh validation

Dataset	Setting	Bugs	LLM Calls	Top-1	Top-3	Top-5	Top-10	MRR@10
Math 21–40	Focused retrieval baseline	20	0	0.7000	0.9500	0.9500	0.9500	0.8167
Math 21–40	Generic selector + DeepSeek	20	3	0.8000	1.0000	1.0000	1.0000	0.8917

Table 11: Compress 1–40 frozen cross-project follow-up validation

Dataset	Setting	Bugs	LLM Calls	Top-1	Top-3	Top-5	Top-10	MRR@10
Compress 1–40	Frozen focused retrieval	40	0	0.7500	0.9250	0.9500	0.9750	0.8299
Compress 1–40	Frozen generic selector + DeepSeek	40	6	0.7750	0.9250	0.9500	0.9750	0.8465

exact path/file, exact component stem, broader component hint, and no target identifier. There are no exact path or filename disclosures. In the 17 records without a direct target identifier, selective reranking still improves MRR@10 from 0.5221 to 0.6216 and Top-10 from 0.8824 to 1.0000. This suggests that the Easy Finance result is not explained only by exact target naming. However, the exact component-stem and broader component-hint rows remain retrospective commit-derived inputs, so the table qualifies the case-study evidence rather than converting it into prospective issue-report validation.

Table 13: Easy Finance clean63 performance by commit-derived query hint category

Query subset	N	BM25 T10	Sel. T10	BM25 MRR@10	Sel. MRR@10	Δ
All clean63	63	0.8730	1.0000	0.5647	0.6729	0.1082
Exact path/file	0	–	–	–	–	–
Exact comp. stem	4	1.0000	1.0000	0.5833	0.5833	0.0000
Component hint	42	0.8571	1.0000	0.5802	0.7022	0.1220
No target id.	17	0.8824	1.0000	0.5221	0.6216	0.0994

BM25 uses `outputs/easy_finance_committed_clean63_bm25_prod_top50.jsonl`. Selective uses `outputs/easy_finance_committed_clean63_bm25_prod_plus_deepseek_selector_v1_ui_evidence_v2.jsonl`. Full Top-k values are in `docs/easy_finance_hint_category_analysis_2026-07-20.md`.

Table 12: Real-world repository case studies

Dataset	Method	Selected Records	Top-1	Top-3	Top-5	Top-10	MRR@10
AboutWork committed-60	BM25 production top50	0 / 60	0.5667	0.8333	0.9167	0.9333	0.6977
AboutWork committed-60	BM25 + selector_v3 + DeepSeek	16 / 60	0.7000	0.9500	0.9667	0.9667	0.8117
Easy Finance clean63	BM25 production top50	0 / 63	0.3968	0.6825	0.8571	0.8730	0.5647
Easy Finance clean63	BM25 + selector_v1 UI evidence v2	10 / 63	0.4921	0.8095	0.9841	1.0000	0.6729

These case-study results indicate that the retrieval–selector–rerank pipeline can be applied beyond curated benchmark bugs. However, the conclusion is intentionally conservative: AboutWork and Easy Finance are case studies, and selector/evidence strategies still require validation on new logs or new commit periods before making broad generalization claims. The remaining AboutWork Top-10 misses are selector false negatives, again indicating that selector recall is a central bottleneck.

6.1.3 Evidence-Strength Summary. The benchmark results provide controlled validation under frozen protocols, while AboutWork and Easy Finance provide case-study evidence. Closure remains the main held-out benchmark, Math provides supporting non-Closure validation, and Compress provides a later frozen test on a previously unused project. The limited Compress change and missed hard cases prevent a broad cross-project superiority claim. The real-world results demonstrate the feasibility of applying the retrieval–selector–rerank pattern in two additional repository settings, but they do not establish out-of-sample selector transfer or broad industrial generalization.

6.2 RQ2: Accuracy–Cost Mechanisms of Selective Reranking

RQ2 explains why selective reranking improves the final ranking in some cases but does not recover all available reranking gain. The experiments in this subsection have different roles. Candidate-recall analysis asks whether the correct file is reachable at all. Selector diagnostics ask whether hard retrieval failures are routed to the LLM. Full rerank-all is a no-selector diagnostic comparison for one-shot reranking under the same candidate pool, not the proposed deployment mode. Repeated Closure reranking measures hosted-model stability under the fixed prompt configuration. Cost in this subsection means model requests, provider-reported token usage, and wall-clock runtime, not a fixed monetary price.

6.2.1 Candidate Recall and Evidence Quality. The remaining failures are dominated by upstream limitations rather than invalid LLM output. Three failure sources explain most of the remaining errors.

First, candidate retrieval misses define a hard upper bound for reranking. If the correct file is absent from the candidate pool, LLM reranking cannot recover it. `Closure-98` is a representative Top-50 retrieval miss in the Closure frozen held-out aggregate.

Second, selector false negatives limit the benefit of selective reranking. In Closure 61–100, there are 15 baseline Top-5 failures, but the selector covers only 6 of them. There are 11 baseline Top-10 failures, but the selector covers only 5. The selected cases benefit from reranking, but selector recall remains a bottleneck.

Third, evidence and snippet quality can determine whether the LLM can recognize the correct file. In the `Closure-4` pilot case, the true file `NamedType.java` was present in the candidate pool at rank 49, but the original snippet did not expose the relevant `handleTypeCycle` and cycle-warning evidence. After improving snippet scoring, compacting repeated stack frames, and adding a type-cycle prompt rule, DeepSeek reranked `NamedType.java` to rank 2. This case is used as method-evolution evidence, not as held-out proof.

To make the evidence-quality claim more concrete, a small follow-up ablation reruns the 11 selected Closure 61–100 cases while holding the candidate pool, selected bug IDs, model alias, and MRR@10 boundary fixed. The intended experimental change is the candidate evidence shown to the one-shot reranker. However, each configuration is a separate hosted-model replay, so model variation can still affect the observed ranks. Table 14 shows that all three reranking configurations improve over the selected-case retrieval baseline, but the improvement is not monotonic with prompt size. In this single-run diagnostic, metadata plus deterministic retrieval reasons produced the highest observed selected-case MRR@10, while the full evidence package used the most tokens and ranked slightly lower. This supports a narrower conclusion: evidence representation affects reranking, but adding more context can also introduce cost and noise. Evidence quality is therefore evaluated through this small selected-case ablation plus diagnostic case analysis, not through a dataset-wide evidence ablation across all datasets.

Because each evidence configuration was evaluated through a separate hosted-model replay and the model exhibits run-to-run variation, the differences in Table 14 are descriptive and cannot be attributed solely to evidence content. In particular, the ordering of the three reranking configurations should be interpreted as a single-run diagnostic rather than a stable ranking of evidence packages.

Recall@K sensitivity further explains the candidate-pool design. Closure 61–100 reaches Recall@50 of 0.9737 and Recall@100 of 1.0000, so expanding from Top-50 to Top-100 recovers only 1 additional bug. Math 21–40, Compress 1–40, AboutWork-60, and Easy Finance reach Recall@50 of 1.0000. This supports Top-50 as a cost/recall trade-off for the current pipeline.

6.2.2 Non-Oracle Selection and Cost Control. Closure 61–100 uses LLM reranking for 11 of 38 bugs, a selected fraction of 0.2895. The selected cases consume 408568 provider-reported tokens and 178.8 seconds, or 37142.5 tokens and 16.3 seconds per selected case on average. At this budget, Top-5 improves by 0.1579, Top-10 improves by 0.1316, and MRR@10 improves by 0.1568. Math 21–40 uses only 3 LLM calls and 106016 tokens, improving Top-10 from 0.9500 to

Table 14: Closure selected-case evidence-package ablation

Evidence package	Req.	Tok./case	T1	T3	T5	T10	MRR@10
Retrieval baseline	0	0.0	0.1818	0.3636	0.4545	0.5455	0.3068
Metadata only	11	18279.5	0.4545	1.0000	1.0000	1.0000	0.6818
Metadata + retrieval reasons	11	26376.0	0.6364	0.9091	1.0000	1.0000	0.7455
Full evidence package	11	37207.7	0.5455	0.9091	1.0000	1.0000	0.7152

All rows evaluate the same 11 selected Closure 61–100 cases. Metadata-only evidence includes candidate paths, ranks/scores, packages, class names, and method names. Metadata + retrieval reasons adds deterministic retrieval evidence. Full evidence additionally includes selected source snippets and triggering-test context. This full-evidence row is a new replay for the ablation and does not replace the main aggregate Closure selective result in Table 9 or its selected-case subset reported in Table 24.

1.0000 and MRR@10 from 0.8167 to 0.8917. Compress 1–40 selects 6 of 40 bugs and uses 247062 tokens over 584.4 recorded seconds; only one selected bug changes rank, producing a 0.0167 MRR@10 increase.

The real-world case studies show the same cost-control pattern. AboutWork committed-60 selects 16 of 60 records for one-shot DeepSeek, improving Top-1 from 0.5667 to 0.7000 and MRR@10 from 0.6977 to 0.8117. Easy Finance clean63 selects 10 of 63 records and improves Top-10 from 0.8730 to 1.0000. These results show that selective reranking can recover useful accuracy with a small selected fraction, but they also make the selector the main source of lost opportunity: unselected difficult cases remain on the retrieval fallback path.

6.2.3 Full Rerank Comparison, Selector Diagnostics, and Run Stability. The full one-shot rerank-all setting is used only as a diagnostic comparison. It quantifies the accuracy available when every case is sent to the LLM, while selective reranking quantifies how much of that gain is retained under a smaller call, token, and runtime budget. The selector is not tuned with these results, and the frozen Closure held-out protocol is not changed.

The diagnostic comparison in Table 15 shows that full rerank-all is more accurate than selective reranking on all three datasets. On Closure 61–100, full rerank-all reaches Top-1 of 0.7368 and MRR@10 of 0.8219, while selective reranking reaches Top-1 of 0.5526 and MRR@10 of 0.6513. Table 16 shows that selective reranking therefore retains 46.15% of the full Top-1 gain and 47.89% of the full MRR@10 gain, while saving 71.05% of calls, 71.25% of tokens, and 74.69% of runtime. This result changes the interpretation of the method: selective reranking is not the most accurate one-shot configuration available under the current evidence package. Instead, selective reranking preserves a substantial part of the reranking gain under a smaller budget, while the gap to full rerank-all measures the empirical cost of selector false negatives.

Full gain is calculated as $M_{\text{full}} - M_{\text{retrieval}}$, selective gain as $M_{\text{selective}} - M_{\text{retrieval}}$, and retention as the selective gain divided by the full gain. For a cost quantity C, call, token, and time savings are calculated as $1 - C_{\text{selective}}/C_{\text{full}}$.

The selector diagnostics in Table 17 show that selector recall is an observed bottleneck. Closure has 15 retrieval Top-5 failures, but only 6 are selected; it has 11 Top-10 failures, but only 5 are selected. Compress provides a sharper transfer diagnostic: neither of its two Top-5 retrieval failures is selected, including the sole Top-10 failure `Compress-35`, whose correct file is available at candidate rank 28. Among selected Closure cases, Table 18 shows that reranking improves 9, leaves 1 unchanged, and degrades 1 case. The degradation is `Closure-82`, where the reciprocal rank changes from 1.0000 to 0.5000. Thus, LLM reranking is not only a recovery mechanism; it can also destabilize a case that retrieval already ranked correctly.

Table 19 shows that repeated Closure selective runs have non-negligible hosted-model variation, but the variation is smaller than the retrieval-to-selective gain for Top-5, Top-10, and MRR@10. The five repeated runs have MRR@10 mean 0.6383 and standard deviation 0.0163, compared with the original retrieval-to-selective MRR@10 gain of 0.1568. The original run remains the main reported result; the repeated runs are robustness evidence and are not cherry-picked. Appendix Table 27 lists the run-level details.

Table 15: Retrieval, full rerank-all, and selective rerank diagnostic comparison

Data	Setting	N	Calls	Sel.	Tokens	Tok./case	Sec.	Sec./case	T1	T3	T5	T10	MRR@10
Closure	Retr.	38	0	0.0000	0	0.0	0.0	0.0	0.3947	0.5263	0.6053	0.7105	0.4945
Closure	Full	38	38	1.0000	1420902	37392.2	706.7	18.6	0.7368	0.9211	0.9737	0.9737	0.8219
Closure	Sel.	38	11	0.2895	408568	37142.5	178.8	16.3	0.5526	0.7105	0.7632	0.8421	0.6513
Math	Retr.	20	0	0.0000	0	0.0	0.0	0.0	0.7000	0.9500	0.9500	0.9500	0.8167
Math	Full	20	20	1.0000	652313	32615.7	395.4	19.8	0.9000	1.0000	1.0000	1.0000	0.9500
Math	Sel.	20	3	0.1500	106016	35338.7	48.9	16.3	0.8000	1.0000	1.0000	1.0000	0.8917
AboutWork	Retr.	60	0	0.0000	0	0.0	0.0	0.0	0.5667	0.8333	0.9167	0.9333	0.6977
AboutWork	Full	60	60	1.0000	1867359	31122.7	1151.8	19.2	0.8833	0.9833	1.0000	1.0000	0.9264
AboutWork	Sel.	60	16	0.2667	519530	32470.6	336.0	21.0	0.7000	0.9500	0.9667	0.9667	0.8117

Data labels abbreviate Closure 61–100, Math 21–40, and AboutWork committed-60. Retr. = retrieval only; Full = full one-shot rerank-all; Sel. = selective one-shot rerank.

Table 16: Accuracy retention and cost saving relative to full rerank-all

Data	Metric	Retr.	Full	Sel.	Full gain	Sel. gain	Retain	Call save	Tok. save	Time save
Closure	Top-1	0.3947	0.7368	0.5526	0.3421	0.1579	0.4615	0.7105	0.7125	0.7469
Closure	Top-5	0.6053	0.9737	0.7632	0.3684	0.1579	0.4286	0.7105	0.7125	0.7469
Closure	Top-10	0.7105	0.9737	0.8421	0.2632	0.1316	0.5000	0.7105	0.7125	0.7469
Closure	MRR@10	0.4945	0.8219	0.6513	0.3274	0.1568	0.4789	0.7105	0.7125	0.7469
Math	Top-1	0.7000	0.9000	0.8000	0.2000	0.1000	0.5000	0.8500	0.8375	0.8764
Math	Top-10	0.9500	1.0000	1.0000	0.0500	0.0500	1.0000	0.8500	0.8375	0.8764
Math	MRR@10	0.8167	0.9500	0.8917	0.1333	0.0750	0.5625	0.8500	0.8375	0.8764
AboutWork	Top-1	0.5667	0.8833	0.7000	0.3167	0.1333	0.4211	0.7333	0.7218	0.7083
AboutWork	Top-10	0.9333	1.0000	0.9667	0.0667	0.0333	0.5000	0.7333	0.7218	0.7083
AboutWork	MRR@10	0.6977	0.9264	0.8117	0.2287	0.1140	0.4984	0.7333	0.7218	0.7083

Retain is the selective gain divided by the full rerank-all gain for the same metric. Call, token, and time savings are calculated against full rerank-all.

Table 17: Selector hard-case coverage

Data	N	Sel.	Frac.	T1 miss	T5 miss	T10 miss	Sel. T5	Sel. T10	SelRec@5	SelRec@10	Unsel. T5	Unsel. T10
Closure	38	11	0.2895	23	15	11	6	5	0.4000	0.4545	9	6
Math	20	3	0.1500	6	1	1	1	1	1.0000	1.0000	0	0
Compress	40	6	0.1500	10	2	1	0	0	0.0000	0.0000	2	1
AboutWork	60	16	0.2667	26	5	4	3	2	0.6000	0.5000	2	2
EF clean63	63	10	0.1587	38	9	8	9	8	1.0000	1.0000	0	0
Mockito 21–30	9	4	0.4444	5	4	3	2	2	0.5000	0.6667	2	1
Mockito 31–38	8	4	0.5000	7	2	1	2	1	1.0000	1.0000	0	0

T5/T10 miss denote retrieval-baseline failures at Top-5/Top-10. Sel. T5/T10 count those failures selected for reranking. SelRec@5 and SelRec@10 are selector recall over those Top-5/Top-10 failure sets. Unsel. T5/T10 are failures left on retrieval fallback.

Table 18: Selected-case effect and degradation

Data	Sel. N	Improved	Degraded	Unchanged	Mean ΔRR	T5+	T5-	T10+	T10-	Notes
Closure	11	9	1	1	0.5417	6	0	5	0	`Closure-82`: RR 1.0000 to 0.5000
Math	3	2	0	1	0.5000	1	0	1	0	–
Compress	6	1	0	5	0.1111	0	0	0	0	`Compress-2`: RR 0.3333 to 1.0000
AboutWork	16	9	0	7	0.4275	3	0	2	0	–
EF clean63	10	10	0	0	0.6815	8	0	8	0	–
Mockito 31–38	4	4	0	0	0.8393	2	0	1	0	–

Improved, Degraded, and Unchanged count selected cases whose reciprocal rank improves, degrades, or stays unchanged after reranking. Mean ΔRR is the mean reciprocal-rank change. T5+/T5- and T10+/T10- count Top-5/Top-10 recoveries and losses.

Table 19: Closure 61–100 selective rerank run-to-run variance

Data	Runs	T1 mean/sd [min,max]	T3 mean/sd [min,max]	T5 mean/sd [min,max]	T10 mean/sd [min,max]	MRR@10 mean/sd [min,max]	Invalid JSON	Mean tokens
Closure	5	0.5421 / 0.0235 [0.5263, 0.5789]	0.6895 / 0.0288 [0.6579, 0.7105]	0.7579 / 0.0118 [0.7368, 0.7632]	0.8421 / 0.0000 [0.8421, 0.8421]	0.6383 / 0.0163 [0.6237, 0.6645]	0.0	413946.4

6.3 RQ3: Agentic and Verifier Ablations

Controlled agentic inspection and verifier reranking are evaluated as RQ3 extensions and ablations. Table 20 reports the Easy Finance strict62, Defects4J diagnostic mini-benchmark, and AboutWork committed-60 comparisons. For Easy Finance and AboutWork, selected records and model requests are separated: one-shot reranking uses one request per selected record, while the agentic setting may issue multiple requests for the same selected record before producing the final ranking.

The Easy Finance agentic trace contains 25 model-request steps across the same 9 selected records. The verifier adds one additional request for each selected record, so the combined agentic-plus-verifier setting uses 34 model requests in total. Top-1, Top-5, and Top-10 remain unchanged, while Top-3 and MRR@10 vary slightly across the three configurations. The cost profile is not monotonic in every field: the agentic-only run uses more requests and tokens but lower recorded wall-clock time than the one-shot run, while the agentic-plus-verifier setting has the largest request count, token count, and runtime. None of these extra-request settings improves all ranking metrics.

The Defects4J mini-benchmark contains 10 known diagnostic cases, including state-reset evidence, type-cycle evidence, pass-chain retrieval boundaries, selector false negatives, utility-file ambiguity, retrieval-boundary negatives, and Mockito pattern generalization. It is used to analyze RQ3 behavior and does not replace the Closure held-out benchmark result.

Per-bug ranks show that one-shot reranking reaches rank 1 on `Math-14`, `Closure-65`, `Mockito-26`, and `Mockito-28`; reaches rank 3 on `Closure-13`; and reaches rank 2 on `Closure-67` and `Closure-75`. Agentic inspection does not add new successes and demotes several one-shot successes. The verifier consumes additional tokens but produces the same ranking as the agentic output.

On AboutWork committed-60, one-shot, agentic, and agentic + verifier have identical per-bug correct ranks. The agentic run uses more model calls and nearly the same recorded runtime as one-shot, but fewer tokens because the agentic requests use shorter per-request contexts than the one-shot evidence package. The verifier adds 16 extra requests, 148718 tokens, and 196.2 seconds without improving aggregate metrics or per-bug ranks. Controlled agentic inspection is therefore technically feasible and produces analyzable traces, but it does not clearly outperform one-shot reranking in the current design. The verifier remains a negative ablation.

Table 20: One-shot, agentic, and verifier results across RQ3 settings

Setting	Method	Sel.	Req.	T1	T3	T5	T10	MRR@10	Tok.	Sec.
Easy Finance strict62	One-shot UI evidence v2	9 / 62	9	0.5000	0.8226	1.0000	1.0000	0.6817	226860	188.5
Easy Finance strict62	Controlled agentic s2	9 / 62	25	0.5000	0.8065	1.0000	1.0000	0.6831	250481	154.4
Easy Finance strict62	Agentic s2 + verifier	9 / 62	34	0.5000	0.8065	1.0000	1.0000	0.6804	331050	276.4
Defects4J diagnostic mini	Retrieval baseline top50	–	0	0.0000	0.0000	0.0000	0.2000	0.0268	0	0.0
Defects4J diagnostic mini	One-shot DeepSeek	–	10	0.4000	0.7000	0.8000	0.9000	0.5644	345937	174.1
Defects4J diagnostic mini	Agentic DeepSeek	–	32	0.3000	0.6000	0.6000	0.8000	0.4768	441952	244.6
Defects4J diagnostic mini	Agentic + verifier DeepSeek	–	42	0.3000	0.6000	0.6000	0.8000	0.4768	554133	370.1
AboutWork committed-60	One-shot DeepSeek	16 / 60	16	0.7000	0.9500	0.9667	0.9667	0.8117	519530	336.0
AboutWork committed-60	Agentic DeepSeek s2	16 / 60	40	0.7000	0.9500	0.9667	0.9667	0.8117	440542	339.6
AboutWork committed-60	Agentic s2 + verifier	16 / 60	56	0.7000	0.9500	0.9667	0.9667	0.8117	589260	535.8

6.4 Supplemental Statistical Analysis

To avoid reporting only proportions, this section adds raw counts, median reciprocal rank at depth 10, standard deviation, paired tests, Recall@K curves, and selected-versus-unselected subset metrics. Tables 21 and 22 report count-level outcomes, dispersion, and paired statistical tests. Because the sample sizes are limited, McNemar and Wilcoxon p-values are interpreted descriptively.

Table 23 reports the Recall@K sensitivity results used to choose the default candidate-pool size. Closure 61–100 reaches Recall@50 of 0.9737; expanding to Top-100 recovers only `Closure-98`. Math, Compress, AboutWork, and Easy Finance already reach Recall@50 of 1.0000. Increasing the candidate pool beyond 50 would raise evidence-construction and prompt cost while providing limited additional recall in the current datasets.

Table 24 uses the same MRR@10 boundary for baseline and final rankings. This matters for unselected fallback cases: because non-selected records keep the retrieval order, their baseline and final MRR@10 values should match. The matching unselected rows in the table are therefore a sanity check that the subset analysis is comparing the same ranking depth.

Table 24 separates selected and unselected subsets. The selected subset results show that the selector often sends difficult cases to the LLM. For example, Closure selected cases improve from Top-5 0.4545 to 1.0000, and Easy Finance selected cases improve from Top-5 0.1000 to 0.9000. The unselected subsets mostly preserve retrieval performance, but remaining misses indicate that selector false negatives remain a central limitation.

Table 21: Raw counts and reciprocal-rank dispersion

Dataset	Method	N	Top-1	Top-3	Top-5	Top-10	MRR@10	Median RR@10	SD(RR@10)
Closure 61–100	Retrieval	38	15/38	20/38	23/38	27/38	0.4945	0.5000	0.4417
Closure 61–100	Selective rerank	38	21/38	27/38	29/38	32/38	0.6513	1.0000	0.4167
Math 21–40	Retrieval	20	14/20	19/20	19/20	19/20	0.8167	1.0000	0.3054
Math 21–40	Selective rerank	20	16/20	20/20	20/20	20/20	0.8917	1.0000	0.2247
Compress 1–40	Retrieval	40	30/40	37/40	38/40	39/40	0.8299	1.0000	0.3089
Compress 1–40	Selective rerank	40	31/40	37/40	38/40	39/40	0.8465	1.0000	0.2993
AboutWork-60	BM25	60	34/60	50/60	55/60	56/60	0.6977	1.0000	0.3666
AboutWork-60	Selective one-shot	60	42/60	57/60	58/60	58/60	0.8117	1.0000	0.3033
Easy Finance clean63	BM25	63	25/63	43/63	54/63	55/63	0.5647	0.5000	0.3827
Easy Finance clean63	Selective one-shot	63	31/63	51/63	62/63	63/63	0.6729	0.5000	0.3360

Table 22: Paired statistical tests

Dataset	Comparison	Wilcoxon p	Mean dRR	Positive / Negative / Zero	McNemar Top-1 p	McNemar Top-5 p
Closure 61–100	Retrieval vs selective	0.0166	+0.1568	9 / 1 / 28	0.0703	0.0312
Math 21–40	Retrieval vs selective	0.1797	+0.0750	2 / 0 / 18	0.5000	1.0000
Compress 1–40	Retrieval vs selective	0.3173	+0.0167	1 / 0 / 39	1.0000	1.0000
AboutWork-60	BM25 vs one-shot	0.0077	+0.1140	9 / 0 / 51	0.0078	0.2500
Easy Finance clean63	BM25 vs one-shot	0.0051	+0.1082	10 / 0 / 53	0.0312	0.0078
Defects4J diagnostic mini	One-shot vs agentic	0.0679	-0.0877	0 / 4 / 6	1.0000	0.5000
AboutWork-60	One-shot vs agentic	1.0000	0.0000	0 / 0 / 60	1.0000	1.0000

Table 23: Retrieval Recall@K sensitivity

Dataset	R@10	R@20	R@50	R@100	R@100 - R@50
Closure 61–100	0.7105	0.8421	0.9737	1.0000	0.0263
Math 21–40	0.9500	0.9500	1.0000	1.0000	0.0000
Compress 1–40	0.9750	0.9750	1.0000	1.0000	0.0000
AboutWork-60	0.9333	0.9667	1.0000	1.0000	0.0000
Easy Finance clean63	0.8730	0.9683	1.0000	1.0000	0.0000
Easy Finance strict62	0.8871	0.9677	1.0000	1.0000	0.0000

Table 24: Selected vs unselected subset metrics

Dataset	Subset	N	Baseline Top-5	Final Top-5	Baseline MRR@10	Final MRR@10
Closure 61–100	selected	11	0.4545	1.0000	0.3068	0.8485
Closure 61–100	unselected	27	0.6667	0.6667	0.5710	0.5710
Math 21–40	selected	3	0.6667	1.0000	0.5000	1.0000
Math 21–40	unselected	17	1.0000	1.0000	0.8725	0.8725
Compress 1–40	selected	6	1.0000	1.0000	0.8889	1.0000
Compress 1–40	unselected	34	0.9412	0.9412	0.8194	0.8194
AboutWork-60	selected	16	0.8125	1.0000	0.5412	0.9688
AboutWork-60	unselected	44	0.9545	0.9545	0.7545	0.7545
Easy Finance clean63	selected	10	0.1000	0.9000	0.0476	0.7292
Easy Finance clean63	unselected	53	1.0000	1.0000	0.6623	0.6623

7 DISCUSSION

7.1 Main Findings

First, the experiments support using LLMs as second-stage rerankers over retrieved candidates, not as replacements for repository search. Retrieval determines whether the correct file is reachable. When the correct file is present and the evidence package exposes relevant failure context, one-shot reranking can move the file to an earlier rank.

Second, selective invocation is a cost-control mechanism, not the highest-accuracy configuration observed in these diagnostics. Full rerank-all is more accurate than selective reranking on Closure, Math, and AboutWork, but it also uses many more calls, tokens, and seconds. The selective method is therefore best read as a budgeted operating point: it retains part of the full-rerank gain while making the cost of LLM invocation explicit.

Third, selector recall is the main reason selective reranking loses accuracy relative to full rerank-all. Closure has 15 retrieval Top-5 failures, but the selector covers only 6 of them; it has 11 Top-10 failures, but the selector covers only 5. The frozen Compress follow-up shows that this problem can become more pronounced under project transfer: the unchanged generic selector covers none of its

two Top-5 failures and misses the sole Top-10 failure even though the correct file is present at candidate rank 28. These missed opportunities explain why selected cases can improve while aggregate performance remains below the available reranking potential.

Fourth, evidence quality remains a separate bottleneck from selection. The `Closure-4` pilot shows that the correct file can be in the candidate pool but still be hard to recognize if the decisive snippet is absent. The selected-case evidence ablation adds a narrower observation: compact metadata plus retrieval reasons can outperform a larger full-evidence package in a single hosted-model replay, so evidence design should be treated as a modeling choice rather than as a simple "more context is better" rule. This means future improvements should not only call the LLM more often; they should also improve which source regions and failure signals enter the evidence package.

Fifth, additional LLM reasoning steps do not automatically improve localization. The agentic and verifier experiments show that tool use and verification increase request counts and can increase token or runtime budgets without improving per-bug correct ranks. If the candidate pool or evidence package is weak, a verifier may simply re-rank the same weak evidence rather than correct the upstream problem.

7.2 Implications for State-of-the-Art Comparison

The interpretation of the results follows the same family-based synthesis used in the related-work comparison and state-of-the-art matrix. The comparison is organized by method mechanisms and evidence assumptions rather than by transferring cross-paper scores.

These results position the thesis closest to retrieval-based bug localization rather than to full repair agents. BugLocator, structured IR, and BLIA motivate the retrieval baseline family: they search repository text and structured bug information, while this thesis keeps retrieval as the recall-producing first stage and studies whether bounded reranking can improve the returned file order [7–9]. The comparison is therefore not that selective reranking replaces IR-based localization, but that it adds a controlled second stage when retrieval evidence is available but not decisive.

Compared with spectrum-based and learning-based fault localization, the main difference is evidence availability. Methods that rely on coverage, spectra, or learned diagnostic dimensions are strong when those artifacts are present, but the case-study records in this thesis are organized around bug reports, failing tests, stack traces, source snapshots, and fixing labels [1, 2, 5]. For Defects4J and AboutWork, the thesis evaluates a pre-fix file-ranking setting rather than claiming superiority over statement-level or spectra-based localization. Easy Finance is evaluated separately under a retrospective commit-derived input boundary.

Compared with recent retrieval-augmented LLM localization systems, the thesis makes a narrower claim. Systems such as SweRank, BLAgent, FaR-Loc, SieveFL, and RGFL show that retrieval, filtering, bounded candidate sets, and LLM-based ranking or reasoning are important state-of-the-art directions [28–32]. BLAgent is especially close because it studies file-level bug localization with agentic RAG, bounded candidate reasoning, and cost-aware evaluation. This thesis does not claim to outperform those systems, because their datasets, granularity, runtime evidence, repair integration, learned components, and agentic protocols differ from the file-level protocol used here. Its contribution is to isolate a specific routing question: how much of the full one-shot reranking gain can a non-oracle per-bug selector retain while reducing calls, tokens, and runtime by allowing some bugs to bypass the LLM and keep deterministic retrieval fallback.

Compared with agentic systems, the main implication is that more model interaction is not automatically better under a file-level ranking protocol. The full rerank-all diagnostics show that one-shot reranking has substantial accuracy potential, while the selective setting shows the cost of imperfect gating [11, 12, 16, 17, 49, 50]. Industry-facing systems such as Project Glasswing and Mythos Preview further show why this question matters in practice: tool-using models are increasingly evaluated for multi-step security investigation and vulnerability discovery, but those reports do not provide direct file-level fault-localization baselines [51, 52]. Those workflows also treat confirmation and triage as part of vulnerability-discovery practice, whereas the verifier studied here is a bounded second pass over the same file-level candidate evidence. The RQ3 ablations therefore test a narrower claim: controlled tool use and verifier reranking do not clearly improve the ranking unless they expose better evidence than the one-shot package. This places the contribution between retrieval-based baselines and repository-scale agents: the thesis supports bounded, evidence-aware LLM use, while leaving broader repair, vulnerability discovery, and unconstrained agentic search outside its claim.

7.3 Error Analysis

Table 25 summarizes the main error categories.

The failures are concentrated in upstream retrieval, evidence construction, and selector coverage, rather than in invalid LLM output. Future improvements should therefore prioritize non-oracle selector recall, candidate recall, and evidence construction instead of simply adding more LLM reasoning steps.

8 THREATS TO VALIDITY

8.1 Internal Validity

Ground truth comes from modified files or classes, which may include non-root-cause changes. This is especially relevant for real-project git-history-derived cases, where a fixing commit may include refactoring, formatting, or opportunistic changes. Prompt construction, snippet selection, and selector design are also part of the method and can influence LLM performance. Closure and Mockito pilot experiments include targeted evidence add-ons, so pilot results are separated from frozen held-out benchmark evidence.

Another internal-validity risk is that LLM reranking can perturb otherwise good retrieval rankings. The selected-case diagnostic finds one Closure selected case, `Closure-82`, where reranking lowers the reciprocal rank from 1.0000 to 0.5000. This does not erase the aggregate improvement, but it shows that the reranker is not monotonic with respect to retrieval quality.

Table 25: Error analysis summary

Error Type	Representative Cases	Cause	Lesson
Recovered evidence/snippet miss	`Closure-4` pilot	Faulty file was present at candidate rank 49, but the original snippet omitted type-cycle evidence.	Evidence construction can determine whether one-shot rerank succeeds.
Selector false negative	`Closure-67`, `Closure-75`, `Closure-88`, `Closure-99`	Correct file was in the candidate pool, but selector did not call the LLM.	Selector recall is the main bottleneck.
Selector false positive / rerank destabilization	`Closure-82`	Retrieval already ranked a correct file first, but the selected rerank moved the first correct file to rank 2.	Selection must also account for cases where reranking can harm an already good retrieval ranking.
Candidate retrieval miss	`Closure-98`	Correct file was outside retrieval Top-50.	Rerank cannot fix absent candidates.
Non-Closure recovered hard case	`Math-31`	Correct file was in candidate pool at rank 26 and selected by state-reset pattern evidence.	Generic pattern selection can transfer beyond Closure.
Cross-project selector miss	`Compress-35`	Correct file was in the Top-50 pool at rank 28, but the frozen generic selector did not route the bug to the LLM.	Candidate recall can transfer while selector recall does not.
Utility-file ambiguity	`Closure-61`, `Closure-75`, `Closure-94`	Failure context points to compiler passes, but fix touches `NodeUtil.java`.	Shared helper files need better non-oracle diagnostics.
Code-output evidence gap	`Closure-65`	`CodePrinter.java` ranked above `CodeGenerator.java`, but selector pattern did not fire.	Code-output selector is useful but too narrow.
Agentic/verifier cost	Easy Finance strict62; Defects4J diagnostic mini; AboutWork committed-60	More LLM steps did not improve evidence quality or per-bug correct ranks.	Keep as RQ3 ablation, not main method.

The evidence-quality ablation is intentionally small. It reruns only the 11 selected Closure 61–100 cases and changes the candidate evidence package while holding the candidate pool and selector fixed. It helps separate metadata, deterministic retrieval reasons, and full source/test context for the selected Closure cases, but it does not establish a dataset-wide or cross-project law about which evidence fields are best. Because each configuration is a separate hosted-model replay, small differences between evidence packages may also reflect run-to-run variation rather than only evidence content.

Easy Finance introduces a separate input-validity limitation. Its bug text is derived retrospectively from fixing-commit metadata rather than from independent pre-fix issue reports. The model does not receive the repair diff, post-fix code, or ground-truth files, but commit-derived text can still contain component or root-cause clues that may not have been available before the fix. The Easy Finance results are therefore interpreted as retrospective git-history case-study evidence rather than as fully prospective fault-localization evidence.

8.2 Construct Validity

The evaluation uses file-level Top-k metrics rather than method-level, statement-level, or patch-level localization. For multi-file bugs, the any-hit file-level definition may overestimate complete localization ability because a bug is counted as successful when any modified file appears in the top-k results. Selective rerank output is truncated to Top-10, so Top-20, Top-50, and Top-100 candidate recall are interpreted only from retrieval baseline outputs.

Cost is also a construct with a specific boundary in this thesis. The reported cost metrics are model requests, provider-reported token usage, and wall-clock runtime. They do not estimate monetary API cost, because provider prices, cache behavior, and hosted-model alias mappings can change after the experiments.

8.3 External Validity

Defects4J is a Java benchmark and does not directly represent all languages or industrial projects. Closure remains the main held-out benchmark source, while Math 21–40 provides non-Closure fresh validation. The later Compress 1–40 follow-up further reduces single-project dependence because the project and protocol were unused during earlier method development. However, its strong retrieval baseline, single changed ranking, and zero selector coverage over its Top-5 failures do not establish stable cross-project improvement. AboutWork committed-60 and Easy Finance are real-project case studies, but their sample sizes are limited and selector/evidence strategies should be validated on new logs or commit periods before making broad industrial claims.

The selector evidence is also dataset-specific. Closure and Math use deterministic selector variants frozen before their reported validation runs, while AboutWork and Easy Finance use case-study selectors associated with the saved real-project runs. Compress uses the unchanged generic selector without project-specific additions, but its failure to select either Top-5 retrieval miss shows that this rule set does not transfer reliably enough to act as a universal selector. The results therefore support the broader retrieval–selector–rerank architecture and its observable accuracy–cost trade-off, but they do not prove that a single universal selector transfers unchanged across projects.

The AboutWork full-rerank diagnostic uses the saved AboutWork selector_v3 evidence configuration from the original case-study run: retrieval evidence is enabled, but triggering-test context is not included. This preserves comparability with the existing AboutWork selective run, but it means the AboutWork full-rerank row is not a fully identical configuration match to the Closure and Math follow-up runs.

8.4 Conclusion Validity

The development and validation slices contain 275 unique usable Defects4J bugs, including the 40-bug Compress follow-up, but this is not a full Defects4J evaluation. Closure frozen held-out validation supports the main method, Math 21–40 provides non-Closure supporting evidence, and Compress tests the frozen generic

pipeline on a previously unused project. Only one Compress record changes rank, yielding Wilcoxon $p = 0.3173$ and McNemar Top-1 $p = 1.0000$; these tests are descriptive and do not support a stable improvement claim. The 10-case Defects4J diagnostic mini-benchmark used for RQ3 reuses bugs from the earlier slices and is constructed around known failure categories, so it is not an unbiased held-out generalization estimate or an additional unique benchmark set. Paired tests are reported as descriptive supporting evidence because the evaluation subsets are limited in size.

Hosted API nondeterminism is another conclusion-validity threat. Repeated DeepSeek calls on the same Closure evidence packages vary across runs. In the five repeated selective runs, Top-1 ranges from 0.5263 to 0.5789 and MRR@10 ranges from 0.6237 to 0.6645. The client supplied `temperature=0.0` and JSON-object response format, but it did not explicitly pin `thinking`, `reasoning_effort`, `top_p`, or `max_tokens`. DeepSeek documentation states that `deepseek-v4-flash` supports thinking and non-thinking modes, that thinking mode is the default, and that `temperature` and `top_p` do not affect generation in thinking mode [57]. The saved usage metadata contains positive reasoning-token counts for many responses, but the saved prediction files do not retain full raw responses or `reasoning_content` for every run. The original run remains the main result, and the variance is reported as robustness evidence rather than selecting the best repeated run. The full rerank-all rows used as denominators in Table 16 were single hosted-model runs rather than repeated runs. Their run-to-run variance is therefore not quantified, so the gain-retention percentages in Table 16 should be interpreted as diagnostic approximations rather than stable point estimates.

Public benchmark contamination is a related hosted-model threat. Defects4J and its project histories are publicly available, so the hosted model may have encountered related source code, bug reports, or fixes during pretraining or system updates. This study cannot directly measure or exclude such exposure. The private case studies reduce this concern for those repositories, but they introduce different reproducibility and data-governance limits.

The full rerank-all diagnostics also constrain the interpretation of selective reranking. Full rerank-all is more accurate than selective reranking on Closure, Math, and AboutWork in the follow-up experiments. Selective reranking should therefore be interpreted as an accuracy–cost trade-off: it saves model calls, tokens, and runtime while retaining part of the full-rerank gain, but it loses accuracy when the selector misses hard cases.

9 REPRODUCIBILITY

The core scripts are:

```
scripts/build_defects4j_dataset.py
scripts/run_hybrid_retrieval.py
scripts/select_rerank_candidates.py
scripts/run_llm_rerank.py
scripts/merge_selective_rerank.py
scripts/evaluate_predictions.py
scripts/summarize_llm_usage.py
scripts/build_rq2_followup_analysis.py
scripts/build_rq2_evidence_ablation_analysis.py
scripts/audit_easy_finance_direct_hints.py
scripts/build_easy_finance_hint_category_analysis.py
scripts/audit_deepseek_reasoning_metadata.py
scripts/build_compress_frozen_analysis.py
```

The main reproduction order is dataset construction, retrieval, selection, selected-case reranking, merge/fallback, evaluation, and usage summarization. The dataset script creates JSONL bug records and evaluation labels from a fixed Defects4J checkout. The retrieval script produces Top-k file rankings. The selector script reads retrieval outputs and bug text to create a non-oracle selection report. The rerank script writes prompts and reranked predictions for selected bug IDs, using either dry-run mode for prompt inspection or DeepSeek for model calls. The merge script combines reranked selected cases with retrieval fallback for non-selected cases. The evaluation scripts compute Top-k and MRR@10 for main comparisons, compute Recall@K from retrieval candidate pools, and summarize token and duration metadata from rerank outputs.

The RQ2 follow-up diagnostics are generated from frozen retrieval outputs and saved evidence packages. The exact full-rerank, variance, evaluation, and usage commands are recorded in:

```
outputs/rq2_followup_2026_06_26/run_manifest.csv
outputs/rq2_followup_2026_06_26/run_manifest.json
```

Each manifest row records the run id, command line, model alias `deepseek-v4-flash`, provider, access date, client-supplied temperature 0.0, JSON-object response format, evidence configuration, output paths, prompt/evidence directories, approximate start and completion timestamps inferred from output timestamps and recorded runtimes, token usage, runtime, invalid JSON count, output checksum, and evidence-package checksum.

The DeepSeek client did not explicitly pin the thinking toggle, reasoning effort, top-p, or maximum output-token parameters. A metadata audit of the saved DeepSeek prediction JSONL files found 1332 records with provider usage and 1045 records with positive reasoning-token metadata, but no saved record retained full reasoning content, system fingerprints, or explicit thinking-mode fields. Therefore, the artifacts record the model alias, provider usage, and run dates, but they do not fully reconstruct the effective provider-default reasoning mode for every call.

The review-ready thesis source and selected supporting artifacts are tracked in the local Git repository under the tag `thesis-ana-review-2026-07-21`. The experimental outputs remain primarily identified by dated protocol files, run manifests, output checksums, and evidence-package checksums; the tag records the thesis source and selected summary artifacts, not the complete private or raw experimental archive.

The main RQ2 follow-up commands have the following form; the manifest contains the exact split-specific commands and bug-id lists:

```
python3 scripts/run_llm_rerank.py \
  --bugs data/defects4j/closure_heldout_61_80.jsonl \
  --bm25 outputs/
      closure_heldout_61_80_hybrid_focused_direct_passchain_typesystem_top50.
      jsonl \
  --out outputs/rq2_followup_2026_06_26/
      closure_61_80_full_rerank_all_deepseek_s12_ctx12000_top50.jsonl \
  --provider deepseek --model deepseek-v4-flash \
  --top-candidates 50 --top-output 10 --max-snippet-lines 12 \
  --include-retrieval-evidence --include-test-context \
  --max-test-context-chars 12000 \
  --prompt-dir outputs/prompts_rq2_followup_2026_06_26/
      closure_61_80_full_rerank_all_s12_ctx12000_top50

python3 scripts/merge_selective_rerank.py \
```

```
  --baseline outputs/
      closure_heldout_61_80_hybrid_focused_direct_passchain_typesystem_top50.
      jsonl \
  --rerank outputs/rq2_followup_2026_06_26/
      closure_61_80_selective_variance_run1_deepseek.jsonl \
  --selection outputs/closure_heldout_61_80_selector_closure_cost_control_v3.
      json \
  --out outputs/rq2_followup_2026_06_26/
      closure_61_80_selective_variance_run1_merged.jsonl \
  --top-output 10

python3 scripts/evaluate_predictions.py --ks 1,3,5,10 --per-bug ...
python3 scripts/summarize_llm_usage.py --pred ...
python3 scripts/build_rq2_followup_analysis.py
```

The generated RQ2 follow-up tables and checksums are stored in:

```
docs/rq2_followup_diagnostics_2026-06-26.md
outputs/rq2_followup_2026_06_26/table_a1_retrieval_full_selective.csv
outputs/rq2_followup_2026_06_26/table_a2_retention_cost_savings.csv
outputs/rq2_followup_2026_06_26/table_b1_selector_hard_case_coverage.csv
outputs/rq2_followup_2026_06_26/table_b2_selected_case_effect_degradation.csv
outputs/rq2_followup_2026_06_26/table_c1_variance_aggregate.csv
outputs/rq2_followup_2026_06_26/table_c2_variance_per_run.csv
outputs/rq2_followup_2026_06_26/artifact_checksums.csv
```

The small RQ2 evidence-quality ablation uses the same Closure 61–100 selected bug IDs and compares metadata-only, metadata-plus-retrieval-reasons, and full-evidence prompt packages. The summary, per-bug ranks, usage files, and checksums are stored in:

```
docs/rq2_evidence_ablation_2026-07-20.md
outputs/rq2_evidence_ablation_2026_07_20/evidence_ablation_summary.csv
outputs/rq2_evidence_ablation_2026_07_20/evidence_ablation_usage.csv
outputs/rq2_evidence_ablation_2026_07_20/evidence_ablation_selected_case_ranks.
      csv
outputs/rq2_evidence_ablation_2026_07_20/evidence_ablation_summary.json
outputs/rq2_evidence_ablation_2026_07_20/evidence_ablation_artifact_checksums.
      csv
```

The Easy Finance direct-target-hint audit is generated from the saved clean63 and strict62 JSONL records and does not rerun retrieval or LLM calls. Its outputs are:

```
docs/easy_finance_direct_hint_audit_2026-07-20.md
outputs/easy_finance_direct_hint_audit_2026_07_20/
      easy_finance_direct_hint_audit_summary.csv
outputs/easy_finance_direct_hint_audit_2026_07_20/
      easy_finance_direct_hint_audit_details.csv
outputs/easy_finance_direct_hint_audit_2026_07_20/
      easy_finance_direct_hint_audit_summary.json
```

The Easy Finance hint-category analysis reuses the direct-hint audit and the existing clean63 prediction files to group BM25 and selective performance by query-hint category. The DeepSeek reasoning-metadata audit scans saved prediction JSONL files and does not rerun model calls. Their outputs are:

```
docs/easy_finance_hint_category_analysis_2026-07-20.md
outputs/easy_finance_hint_category_analysis_2026_07_20/
      easy_finance_hint_category_metrics.csv
outputs/easy_finance_hint_category_analysis_2026_07_20/
      easy_finance_hint_category_metrics.json
docs/deepseek_reasoning_metadata_audit_2026-07-20.md
outputs/deepseek_reasoning_metadata_audit_2026_07_20/
      deepseek_reasoning_metadata_by_file.csv
outputs/deepseek_reasoning_metadata_audit_2026_07_20/
      deepseek_reasoning_metadata_summary.json
```

Future reruns should explicitly set the thinking-mode toggle, reasoning effort, and maximum output-token limit, and should retain the effective response metadata when the provider returns it. This would avoid relying on mutable hosted-model defaults and would reduce the risk of truncated JSON output.

The Compress 1–40 cross-project follow-up is documented independently from the June validation campaign. Its frozen protocol records the pre-run project boundary, generic selector settings, script checksums, model configuration, and the network-timeout amendment. The generated report and machine-readable summaries are:

```
docs/frozen_protocol_2026-08-11_compress_1_40.md
docs/compress_frozen_1_40_validation_report_2026-08-11.md
outputs/compress_frozen_1_40_analysis_2026_08_11/analysis_manifest.json
outputs/compress_frozen_1_40_analysis_2026_08_11/metric_distributions.csv
outputs/compress_frozen_1_40_analysis_2026_08_11/paired_tests.csv
outputs/compress_frozen_1_40_analysis_2026_08_11/recall_curves.csv
outputs/compress_frozen_1_40_analysis_2026_08_11/selector_subset_metrics.csv
outputs/compress_frozen_1_40_analysis_2026_08_11/artifact_checksums.csv
```

The key protocol and result documents are:

```
docs/frozen_protocol_2026-06-01.md
docs/frozen_protocol_2026-06-02_closure_81_100.md
docs/closure_heldout_61_80_validation_report.md
docs/closure_heldout_81_100_validation_report.md
docs/closure_frozen_heldout_61_100_summary.md
docs/frozen_protocol_2026-06-10_math_21_40.md
docs/math_21_40_fresh_validation_report_2026-06-10.md
docs/frozen_protocol_2026-08-11_compress_1_40.md
docs/compress_frozen_1_40_validation_report_2026-08-11.md
docs/statistical_supplement_2026-06-10.md
docs/rq2_followup_diagnostics_2026-06-26.md
docs/rq2_evidence_ablation_2026-07-20.md
docs/easy_finance_direct_hint_audit_2026-07-20.md
docs/easy_finance_hint_category_analysis_2026-07-20.md
docs/deepseek_reasoning_metadata_audit_2026-07-20.md
docs/aboutwork_committed_60_rerank_results_2026-06-08.md
docs/aboutwork_committed_60_agentic_verifier_results_2026-06-08.md
docs/thesis_consistency_review_2026-06-03.md
```

Reproduction requires a local Defects4J checkout, corresponding bug workspaces, a DeepSeek API key, and the fixed retrieval, selector, and rerank parameters documented in the frozen protocols.

10 CONCLUSION

This thesis studied selective evidence-aware LLM reranking for file-level fault localization. The central idea is to keep repository search deterministic and inexpensive, then invoke an LLM only when the available input evidence suggests that retrieval alone may be insufficient. The thesis does not claim novelty for retrieval-augmented reranking itself; its primary contribution is the controlled evaluation of a non-oracle per-bug invocation policy with deterministic retrieval fallback. The primary benchmark and AboutWork evaluations use pre-fix evidence, whereas Easy Finance uses retrospective commit-derived text. The resulting pipeline combines focused hybrid retrieval, structured evidence construction, a non-oracle selective rerank gate, one-shot DeepSeek reranking, and retrieval fallback.

The main empirical result is that selective reranking can improve file-level rankings when the correct file is present in the candidate pool and the evidence package exposes useful signals. On the frozen Closure 61–100 held-out slice, selective DeepSeek reranking improves Top-1, Top-5, Top-10, and MRR@10 while invoking the LLM for 11 of 38 bugs. Math 21–40 provides supporting non-Closure evidence: the retrieval baseline is already strong, but selective reranking closes the remaining Top-10 miss and improves MRR@10 with only 3 LLM calls. A frozen follow-up on 40 previously unused Compress bugs produces a smaller result: one rank

improves, Top-1 rises from 0.7500 to 0.7750, MRR@10 rises from 0.8299 to 0.8465, and Top-3, Top-5, and Top-10 remain unchanged. The generic selector also misses both Compress Top-5 retrieval failures. The AboutWork and Easy Finance case studies show the feasibility of applying the same retrieval–selector–rerank pattern to real-project records, although these results should be interpreted as case-study evidence rather than broad industrial generalization. Easy Finance is additionally retrospective because its bug text is derived from fixing-commit metadata, so it does not provide the same prospective input boundary as Defects4J or AboutWork.

The follow-up full rerank-all diagnostics sharpen this conclusion. Sending every case to DeepSeek obtains higher accuracy than selective reranking on Closure, Math, and AboutWork, but it also consumes substantially more calls, tokens, and runtime. Selective reranking is therefore best understood as a cost-control mechanism that retains part of the full-rerank gain, not as a method that always matches full-rerank accuracy.

The experiments also clarify where the method fails. LLM reranking cannot recover a file that retrieval did not include in the candidate pool. Selective reranking can miss useful opportunities when the selector fails to identify a difficult case; the Compress follow-up shows that this limitation persists under an unchanged cross-project selector. Evidence construction also affects ranking: the Closure diagnostic case shows that missing snippets can hide decisive failure evidence, while the small selected-case ablation shows that adding more source and test context is not automatically better than a more compact evidence package with retrieval reasons. These findings shift the improvement target away from simply adding more model calls and toward better candidate recall, better selector recall, and better evidence construction.

The agentic and verifier experiments provide a negative ablation result. Controlled agentic inspection is technically feasible and can produce auditable traces, but it does not clearly outperform one-shot reranking in the current experiments. Verifier reranking adds cost without improving the reported rankings. For the file-level task studied here, additional LLM interaction is not automatically valuable unless it improves the quality of the evidence available to the ranking decision.

Future work should first strengthen the non-oracle selector. The current selector is useful for cost control, but false negatives remain a central bottleneck. A more reliable selector should detect utility-file ambiguity, code-output evidence, state-reset patterns, and other difficult retrieval cases without using ground-truth labels. Second, evidence construction should be improved so that snippets expose the local state transitions, failing assertions, stack-relevant code, and domain-specific signals that make a candidate file suspicious. Third, the evaluation should be extended to additional projects, languages, and real-world bug streams, while preserving the same no-leakage protocol and testing whether one selector can transfer across projects instead of relying on dataset-specific selector rules. Finally, future systems could connect file-level localization to downstream repair or developer workflows, but that should be evaluated separately from the ranking task studied in this thesis.

Overall, the thesis supports a conservative conclusion: LLMs are useful for fault localization when they are used as bounded, evidence-aware rerankers rather than as unconstrained repository search agents. Selective invocation keeps cost visible, retrieval fallback preserves deterministic behavior, and the remaining failures point mainly to upstream evidence and selection quality.

A ADDITIONAL TABLES AND PER-BUG DIAGNOSTICS

The main text reports aggregate metrics, raw-count summaries, selected-versus-unselected behavior, and a compact error-analysis table. Longer per-bug outputs are not duplicated here because they are easier to audit in the generated result files and diagnostic reports. Table 26 lists the main artifacts that contain per-bug ranks, selected-case behavior, selector false negatives, and RQ3 diagnostic traces.

The artifact name `docs/rq5_defects4j_mini_benchmark_results_2026-06-03.md` is historical: it was generated before the thesis research questions were consolidated. In the revised thesis structure, this diagnostic mini-benchmark is discussed under RQ3.

Table 26: Per-bug diagnostic artifacts

Artifact	Diagnostic Role
Closure held-out summary `docs/closure_frozen_heldout_61_100_summary.md`	Closure held-out aggregate, selector coverage, retrieval miss summary, and source artifacts.
Math validation report `docs/math_21_40_fresh_validation_report_2026-06-10.md`	Math fresh-validation selected cases, rank changes, Recall@K, and token usage.
Compress frozen follow-up `docs/compress_frozen_1_40_validation_report_2026-08-11.md`	Previously unused Defects4J project, frozen generic-selector result, cost, paired statistics, and selector false negatives.
Statistical supplement `docs/statistical_supplement_2026-06-10.md`	Raw counts, reciprocal-rank dispersion, paired tests, Recall@K, and selected/unselected subset metrics.
RQ2 follow-up diagnostics `docs/rq2_followup_diagnostics_2026-06-26.md`	RQ2 full rerank-all comparison, selector diagnostics, run-to-run variance, and follow-up artifact paths.
RQ2 evidence ablation `docs/rq2_evidence_ablation_2026-07-20.md`	Closure selected-case evidence-package ablation, including metadata-only, metadata-plus-retrieval-reasons, and full-evidence runs.
Easy Finance direct-hint audit `docs/easy_finance_direct_hint_audit_2026-07-20.md`	Commit-derived query audit for exact path/file disclosures, exact component-stem phrases, component-level hints, and no-target-identifier records.
Easy Finance hint-category analysis `docs/easy_finance_hint_category_analysis_2026-07-20.md`	Existing clean63 prediction results grouped by direct-hint category, including the no-target-identifier subset.
DeepSeek reasoning metadata audit `docs/deepseek_reasoning_metadata_audit_2026-07-20.md`	Saved DeepSeek output metadata audit for reasoning-token counts and missing explicit thinking-mode fields.
AboutWork one-shot report `docs/aboutwork_committed_60_rerank_results_2026-06-08.md`	AboutWork selected-case behavior, remaining errors, and one-shot rerank usage.
AboutWork agentic/verifier report `docs/aboutwork_committed_60_agentic_verifier_results_2026-06-08.md`	AboutWork one-shot, agentic, and verifier comparison with usage summaries.
Defects4J RQ3 diagnostic mini `docs/rq5_defects4j_mini_benchmark_results_2026-06-03.md`	Defects4J RQ3 diagnostic mini-benchmark, including per-bug ranks for one-shot, agentic, and verifier variants.
Error analysis table `docs/error_analysis_table_2026-06-03.md`	Compact error categories and representative cases used to build the thesis discussion.
Claim-evidence audit `writing/CLAIM_EVIDENCE_AUDIT.md`	Claim-evidence map connecting thesis claims to local result reports and implementation files.

Table 27: Closure selective rerank repeated runs

Run	Calls	Tokens	Runtime	T1	T3	T5	T10	MRR@10	Notes
Original	11	408568	178.8	0.5526	0.7105	0.7632	0.8421	0.6513	Main reported run
Run1	11	414621	238.6	0.5526	0.7105	0.7632	0.8421	0.6425	Repeated API call
Run2	11	413333	238.4	0.5263	0.6579	0.7632	0.8421	0.6237	Repeated API call
Run3	11	415820	257.7	0.5263	0.7105	0.7632	0.8421	0.6338	Repeated API call
Run4	11	412649	221.4	0.5789	0.7105	0.7632	0.8421	0.6645	Repeated API call
Run5	11	413309	226.8	0.5263	0.6579	0.7368	0.8421	0.6272	Repeated API call

REFERENCES

[1] James A. Jones, Mary Jean Harrold, and John T. Stasko. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering*, pages 467–477, New York, NY, USA, 2002. Association for Computing Machinery. doi: 10.1145/581339.581397.

[2] Rui Abreu, Peter Zoeteweij, Rob Golsteijn, and Arjan J. C. van Gemund. A practical evaluation of spectrum-based fault localization. *Journal of Systems and Software*, 82(11):1780–1792, 2009. doi: 10.1016/j.jss.2009.06.035.

[3] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8): 707–740, 2016. doi: 10.1109/TSE.2016.2521368.

[4] Qusay Idrees Sarhan and Arpad Beszedes. A survey of challenges in spectrum-based software fault localization. *IEEE Access*, 10:10618–10639, 2022. doi: 10.1109/ACCESS.2022.3144079.

[5] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 169–180, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.1145/3293882.3330574.

[6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

[7] Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering*, pages 14–24, Los Alamitos, CA, USA, 2012. IEEE Computer Society. doi: 10.1109/ICSE.2012.6227210.

[8] Ripon K. Saha, Matthew Lease, Sarfraz Khurshid, and Dewayne E. Perry. Improving bug localization using structured information retrieval. In *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering*, pages 345–355, Los Alamitos, CA, USA, 2013. IEEE Computer Society. doi: 10.1109/ASE.2013.6693093.

[9] Klaus Changsun Youm, June Ahn, and Eunseok Lee. Improved bug localization based on code change histories and bug reports. *Information and Software Technology*, 82:177–192, 2017. doi: 10.1016/j.infsof.2016.11.002.

[10] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. Large language models in fault localisation, 2023. URL https://arxiv.org/abs/2308.15276.

[11] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent J. Hellendoorn. Large language models for test-free fault localization. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 17:1–17:12, New York, NY, USA, 2024. Association for Computing Machinery. doi: 10.1145/3597503.3623342.

[12] Sungmin Kang, Gabin An, and Shin Yoo. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering*, 1(FSE):1424–1446, 2024. doi: 10.1145/3660771.

[13] Suhwan Ji, Sanghwa Lee, Changsup Lee, Yo-Sub Han, and Hyeonseung Im. Impact of large language models of code on fault localization. In *Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation*, pages 302–313, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10989036.

[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.139.

[15] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 7212–7225, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.499.

[16] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. AgentFL: Scaling LLM-based fault localization to project-level context, 2024. URL https://arxiv.org/abs/2403.16362.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37*, pages 50528–50652, Red Hook, NY, USA, 2024. Curran Associates, Inc. doi: 10.52202/079017-1601.

[18] Robert Feldt, Sungmin Kang, Juyeon Yoon, and Shin Yoo. Towards autonomous testing agents via conversational large language models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, pages 1688–1693, Los Alamitos, CA, USA, 2023. IEEE Computer Society. doi: 10.1109/ASE56229.2023.00148.

[19] Islem Bouzenia and Michael Pradel. You name it, i run it: An LLM agent to execute tests of arbitrary projects. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1054–1076, 2025. doi: 10.1145/3728922.

[20] Rene Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pages 437–440, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2610384.2628055.

[21] Leonhard Applis, Matthias Pall Gissurarson, and Annibale Panichella. Suspicious types and bad neighborhoods: Filtering spectra with compiler information. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 233–243, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10988920.

[22] Dawn Lawrie and Dave Binkley. On the value of bug reports for retrieval-based bug localization. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*, pages 524–528, Los Alamitos, CA, USA, 2018. IEEE Computer Society. doi: 10.1109/ICSME.2018.00048.

[23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? The Twelfth International Conference on Learning Representations (ICLR), 2024. URL https://openreview.net/forum?id=VTF8yNQM66. Conference paper. The official citation does not assign page numbers.

[24] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. Systematic literature reviews in software engineering: A systematic literature review. *Information and Software Technology*, 51(1):7–15, 2009. doi: 10.1016/j.infsof.2008.09.009.

[25] Stacy K. Lukins, Nicholas A. Kraft, and Letha H. Etzkorn. Source code retrieval for bug localization using latent dirichlet allocation. In *Proceedings of the 15th Working Conference on Reverse Engineering*, pages 155–164, Los Alamitos, CA, USA, 2008. IEEE Computer Society. doi: 10.1109/WCRE.2008.33.

[26] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024. doi: 10.1145/3695988.

[27] Moumita Asad, Rafed Muhammad Yasir, and Sam Malek. Towards explorative IRBL: Combining semantic retrieval with LLM-driven iterative code exploration, 2026. URL https://arxiv.org/abs/2508.00253. arXiv preprint; ISSTA 2026 research paper.

[28] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. SweRank: Software issue localization with code ranking. arXiv preprint arXiv:2505.07849, 2025. URL https://arxiv.org/abs/2505.07849. ICLR 2026 camera-ready version.

[29] Md Afif Al Mamun and Gias Uddin. BLAgent: Agentic RAG for file-level bug localization. *ACM Transactions on Software Engineering and Methodology*, 2026. doi: 10.1145/3830238. URL https://doi.org/10.1145/3830238. Accepted for publication; arXiv:2605.17965.

[30] Xinyu Shi, Zhenhao Li, and An Ran Chen. Enhancing LLM-based fault localization with a functionality-aware retrieval-augmented generation framework. arXiv preprint arXiv:2509.20552, 2025. URL https://arxiv.org/abs/2509.20552.

[31] Mahdi Farzandway and Fatemeh Ghassemi. SieveFL: Hierarchical runtime-aware pruning for scalable LLM-based fault localization. arXiv preprint arXiv:2605.13491, 2026. URL https://arxiv.org/abs/2605.13491.

[32] Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. RGFL: Reasoning guided fault localization for automated program repair using large language models. arXiv preprint arXiv:2601.18044, 2026. URL https://arxiv.org/abs/2601.18044.

[33] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 643–653, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2635868.2635920.

[34] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology*, 31(1):1–74, 2022. doi: 10.1145/3476105.

[35] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. iDFlakies: A framework for detecting and partially classifying flaky tests. In *Proceedings of the 12th IEEE International Conference on Software Testing, Verification and Validation*, pages 312–322, Los Alamitos, CA, USA, 2019. IEEE Computer Society. doi: 10.1109/ICST.2019.00038.

[36] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. Flakeflagger: Predicting flakiness without rerunning tests. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*, pages 1572–1584, Los Alamitos, CA, USA, 2021. IEEE Computer Society. doi: 10.1109/ICSE43902.2021.00140.

[37] Sakina Fatima, Taher A. Ghaleb, and Lionel C. Briand. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Transactions on Software Engineering*, 49(4):1912–1927, 2023. doi: 10.1109/TSE.2022.3201209.

[38] Amal Akli, Guillaume Haben, Sarra Habchi, Mike Papadakis, and Yves Le Traon. Flakycat: Predicting flaky tests categories using few-shot learning. In *Proceedings of the 4th ACM/IEEE International Workshop on Automated Software Testing*, pages 140–151, Los Alamitos, CA, USA, 2023. IEEE Computer Society. doi: 10.1109/AST58925.2023.00018.

[39] Riddhi More and Jeremy S. Bradbury. An analysis of llm fine-tuning and few-shot learning for flaky test detection and classification. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 349–359, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10988995.

[40] Sakina Fatima, Hadi Hemmati, and Lionel C. Briand. FlakyFix: Using large language models for predicting flaky test fix categories and test code repair. *IEEE Transactions on Software Engineering*, 50(12):3146–3171, 2024. doi: 10.1109/TSE.2024.3472476.

[41] Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. Why do record/replay tests of web applications break? In *Proceedings of the 9th IEEE International Conference on Software Testing, Verification and Validation*, pages 180–190, Los Alamitos, CA, USA, 2016. IEEE Computer Society. doi: 10.1109/ICST.2016.16.

[42] Zhuolin Xu, Qiushi Li, and Shin Hwei Tan. Understanding and enhancing attribute prioritization in fixing web UI tests with LLMs. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 326–337, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10989008.

[43] Ahmadreza Saboor Yaraghi, Darren Holden, Nafiseh Kahani, and Lionel C. Briand. Automated test case repair using language models. *IEEE Transactions on Software Engineering*, 51(4):1104–1133, 2025. doi: 10.1109/TSE.2025.3541166.

[44] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. Understanding the effectiveness of large language models in detecting security vulnerabilities. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 103–114, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10988968.

[45] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David A. Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we? In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, pages 1729–1741, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICSE55347.2025.00038.

[46] Bozhi Wu, Chengjie Liu, Zhiming Li, Yushi Cao, Jun Sun, and Shang-Wei Lin. Enhancing vulnerability detection via inter-procedural semantic completion. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1–23, 2025. doi: 10.1145/3728912.

[47] Guru Bhandari, Amara Naseer, and Leon Moonen. CVEFixes: Automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, pages 30–39, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3475960.3475985.

[48] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32*, pages 10197–10207, Red Hook, NY, USA, 2019. Curran Associates, Inc.

[49] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. arXiv preprint arXiv:2407.01489, 2024. URL https://arxiv.org/abs/2407.01489.

[50] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. arXiv preprint arXiv:2503.09089, 2025. URL https://arxiv.org/abs/2503.09089.

[51] Anthropic. Project Glasswing. https://www.anthropic.com/glasswing, 2026. Accessed 28 April 2026.

[52] Nicholas Carlini, Newton Cheng, Keane Lucas, Michael Moore, Milad Nasr, Vinay Prabhushankar, Winnie Xiao, Hakeem Angulu, Evyatar Ben Asher, Jackie Bow, Keir Bradwell, Ben Buchanan, David Forsythe, Daniel Freeman, Alex Gaynor, Xinyang Ge, Logan Graham, Kyla Guru, Hasnain Lakhani, Matt McNiece, Mojtaba Mehrara, Renee Nichol, Adnan Pirzada, Sophia Porter, Andreas Terzis, and Kevin Troy. Assessing claude Mythos Preview's cybersecurity capabilities. Anthropic Frontier Red Team Technical Report, 2026. URL https://www.anthropic.com/research/mythos-preview. Accessed 19 July 2026.

[53] Xiaoxiao Gan, Huayu Liang, and Chris Brown. Challenges, strategies, and impacts: A qualitative study on UI testing in CI/CD processes from GitHub developers' perspectives. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 186–197, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10988972.

[54] Lena Gregor, Anja Hentschel, Leon Kastner, and Alexander Pretschner. A taxonomy of integration-relevant faults for microservice testing. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 138–149, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10989000.

[55] Daniel Rodriguez-Cardenas, Safwat Ali Khan, Prianka Mandal, Adwait Nadkarni, Kevin Moran, and Denys Poshyvanyk. Testing practices, challenges, and developer perspectives in open-source IoT platforms. In *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation*, pages 290–301, Los Alamitos, CA, USA, 2025. IEEE Computer Society. doi: 10.1109/ICST62969.2025.10988986.

[56] Amir Makhshari and Ali Mesbah. IoT bugs and development challenges. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*, pages 460–472, Los Alamitos, CA, USA, 2021. IEEE Computer Society. doi: 10.1109/ICSE43902.2021.00051.

[57] DeepSeek-AI. DeepSeek API Documentation: Models, thinking mode, and json output. https://api-docs.deepseek.com/, 2026. Model and pricing documentation accessed 25 June 2026 for the reported experiments; thinking-mode and JSON-output documentation checked 20 July 2026. Model alias recorded in experiment manifests: `deepseek-v4-flash`.